

Major Project Report on
UVION - an AI-Powered Autonomous Farming System
A report submitted in partial fulfilment of the requirements for the Award of Degree of
BACHELOR OF ENGINEERING
in
ARTIFICIAL INTELLIGENCE AND DATA SCIENCE
by
MUSKAAN TABASSUM (1604-22-747-004)
SYEDA FATIMA HUSSAINI (1604-22-747-008)

Under the Guidance of

Mrs. SARA TABASSUM
Assistant Professor



**DEPARTMENT OF COMPUTER SCIENCE AND ARTIFICIAL
INTELLIGENCE**
**MUFFAKHAM JAH COLLEGE OF ENGINEERING AND
TECHNOLOGY**

(Affiliated to Osmania University)

Mount Pleasant, 8-2-249, Road No. 3, Banjara Hills, Hyderabad 500034, Telangana, State, India

YEAR OF SUBMISSION: 2025– 2026



MUFFAKHAM JAH COLLEGE OF ENGINEERING & TECHNOLOGY

(Estd. by Sultan-Ul-Uloom Education Society in 1980)

(Affiliated to Osmania University, Hyderabad)

Approved by the AICTE & Accredited by the NAAC with Grade A+

CERTIFICATE

This is to certify that the **Major Project Report** on **UVION - an AI-Powered Autonomous Farming System** submitted by **MUSKAAN TABASSUM (1604-22-747-004)** and **SYEDA FATIMA HUSSAINI (1604-22-747-008)**, is work done by them under our guidance and submitted during 2025-2026 academic year, in partial fulfilment of the requirement for the Award of the Degree, **Bachelor of Engineering in Artificial Intelligence and Data Science**.
This work is not submitted elsewhere for a degree.

Internal Guide

Mrs. Sara Tabassum
Assistant Professor
CS&AI Dept., MJCET

Prof. Uma N. Dulhare

Head
CS&AI Dept, MJCET

External Examiner

DECLARATION

This is to certify that the work reported in the Major Project Report entitled **UVION - an AI-Powered Autonomous Farming System** is a record of work done by **MUSKAAN TABASSUM (1604-22-747-004)** and **SYEDA FATIMA HUSSAINI (1604-22-747-008)**, in the Department of **Computer Science and Artificial Intelligence, Muffakham Jah College of Engineering and Technology, Osmania University**, the report is based on the Major Project work done entirely by our team and not copied from any other source.

By

**MUSKAAN TABASSUM
(1604-22-747-004)**

**SYEDA FATIMA HUSSAINI
(1604-22-747-008)**

ACKNOWLEDGEMENT

It is indeed with a great sense of pleasure and immense gratitude that we acknowledge the help of these individuals.

Firstly, we would like to thank our Head of the Department and Major Project Coordinator, Prof. Uma N. Dulhare, for her constructive criticism throughout Major Project.

We would like to thank our Major Project Guide, Mrs. Sara Tabassum, Assistant Professor, Department of Computer Science and Artificial Intelligence, for her support in accomplishing the objectives of Major Project.

Finally, we would like to take this opportunity to thank our family for their support throughout Major Project. We sincerely acknowledge and thank all those who have directly or indirectly supported us in the successful completion of Major Project.

**MUSKAAN TABASSUM
(1604-22-747-004)**

**SYEDA FATIMA HUSSAINI
(1604-22-747-008)**

TABLE OF CONTENTS

CONTENT	PAGE NOS
I. ABSTRACT	iii
II. LIST OF FIGURES	iv
III. LIST OF TABLES	v
IV. ACRONYMS	vi
CHAPTERS	
1. INTRODUCTION	1-3
1.1 Introduction	1
1.2 Aim & Objectives	1
1.3 Reason for Project	2
1.4 Problem Statement	3
1.5 Scope	3
1.6 Summary	3
2. LITERATURE SURVEY	4-6
2.1 Survey of Related Work	4
2.2 Benefits of Project	6
3. EXISTING SYSTEM	7-9
3.1 Introduction	7
3.1.1 Disease Detection	7
3.1.2 Fertilizer Recommendation Tools	8
3.2 Key Challenges	9
3.2 Problem Statement	9
4. PROPOSED SYSTEM	10-12
4.1 Introduction	10
4.2 Advantages of Proposed Solution	10
4.3 Specifications of the Proposed System	11
4.3.1 Disease Detection Module	11
4.3.2 Fertilizer Recommendation Module	11
4.3.3 Crop Yield Prediction Module	12
4.3.4 Dataset Integration and Processing	12
4.3.5 System Architecture	12
5. SYSTEM ANALYSIS	13-19
5.1 Introduction	13
5.2 Feasibility Study	13
5.2.1 Technical Feasibility	14
5.2.2 Operational Feasibility	14
5.2.3 Economical Feasibility	14
5.2.4 COCOMO Model Estimation	14
5.2.5 COCOMO II Model	16
5.2.6 Legal Feasibility	17
5.3 System Implementation	18
5.4 Functional Requirements	18
5.5 Non-Functional Requirements	19
5.6 Hardware & Software Requirements	19
6. SYSTEM DESIGN	20-26
6.1 System Architecture Design	20
6.1.1 Data Flow Diagram	21
6.1.2 E-R Diagram	22

6.2 UML Diagrams	23
6.2.1 Usecase Diagram	23
6.2.2 Sequence Diagram	24
6.2.3 Class Diagram	25
6.2.4 Activity Diagram	26
7. METHODOLOGY	27-32
7.1 System Approach	27
7.2 Algorithm	29
7.3 Working process	30
7.3.1 Data Processing and Module Execution	30
7.3.2 Output Generation	32
8. DATASET DESCRIPTION	33-36
8.1 Disease Detection Dataset	33
8.2 Yield Prediction Dataset	34
8.3 Fertilizer Recommendation	35
8.4 Weather Dataset	35
8.5 Dataset Integration Strategy	35
9. MODULE IMPLEMENTATION	37-46
9.1 Disease Detection Model	37
9.1.1 Data Preparation	37
9.1.2 Model Building and Training	37
9.2 Yield Prediction Model Implementation	42
9.3 Fertilizer Recommendation Model Implementation	43
9.4 Results	43
10. TESTING	47-51
10.1 Introduction	47
10.2 Levels of Testing	47
10.2.1 Unit Testing	48
10.2.2 Integration Testing	48
10.2.3 System Testing (End-To-End Testing)	49
10.2.4 User Acceptance Testing (Uat)	50
10.2.5 Performance and Security Testing	50
10.3 Summary	51
11. SCREENSHOTS	52-58
11.1 Login Page	52
11.2 Farm Setup Page	52
11. 3 Dashboard	53
11.4 Health Hub Dashboard	54
11.5 Disease Detection	55
11.6 Fertilizer Recommendation	56
11.7 Yeild Prediction	57
12 CONCLUSION	59-60
12.1 Conclusion	59
12.2 Future Work	59
REFERENCES	61-62

I. ABSTRACT

UVION is an AI-powered, multi-agent autonomous farming system designed to support farmers with intelligent, real-time decision-making across the entire crop lifecycle. The system integrates five specialized agents—Growth Prediction, Disease Detection, Weather & Irrigation, Fertilizer Recommendation, and Yield Prediction—to provide accurate, data-driven insights. UVION uses machine learning models such as LSTMs and CNNs, along with a RAG-based knowledge engine, to forecast crop growth, identify diseases from images, recommend nutrient plans, predict rainfall-driven irrigation needs, and estimate final yield. A simulated IoT sensor environment allows the system to function without hardware dependency, enabling zero-cost deployment. By orchestrating multiple agents through a unified FastAPI backend and presenting outputs on an intuitive React dashboard, UVION delivers an end-to-end intelligent farming assistant that enhances productivity, reduces crop loss, and supports sustainable agricultural practices.

Keywords

AI in Agriculture, Multi-Agent System, Growth Prediction, Disease Detection, Weather Forecasting, Fertilizer Recommendation, Yield Prediction, Time-Series Forecasting, Computer Vision, RAG (Retrieval-Augmented Generation), Autonomous Farming, Smart Agriculture.

II. LIST OF FIGURES

FIGURE NO.	NAME OF THE FIGURE	PAGE NO.
Fig. 6.1	System Architecture	20
Fig 6.1.1	Data Flow Diagram	21
Fig 6.1.2	E-R Diagram	22
Fig 6.2.1	Use case Diagram	23
Fig 6.2.2	Sequence Diagram	24
Fig 6.2.4	Class Diagram	26
Fig 6.2.4	Activity Diagram	27
Fig 7.1	System Approach	29
Fig 7.2	Deep learning model	32
Fig 8.1	Dataset Images	34
Fig 9.1	Code For Data Splitting	38
Fig 9.2	Model Architecture (MobileNetV2)	39
Fig 9.3	Model Functionalities	40
Fig 9.4	Phase1 training	40
Fig 9.5	Phase1 Training Generated Epochs	41
Fig 9.6	Phase2 Training	42
Fig 9.7	Phase2 training output	43
Fig 9.8	Classification Report	45
Fig 9.9	Confusion Matrix	46
Fig 9.10	Accuracy Metrics	47
Fig 11.1	Login Page	52
Fig 11.2	Farm setup page	53
Fig 11.3	Dashboard	54
Fig 11.4	Health Hub Dashboard	55
Fig 11.5	Disease Detection	56
Fig 11.6	Fertilizer Recommendation	57
Fig 11.7	Yeild Prediction	58

III. LIST OF TABLES

TABLE NO.	TABLE NAME	PAGE NO.
Table 2.1	Survey of Related Work	4
Table 5.6.1	Hardware Requirements	19
Table 5.6.2	Software Requirements	19
Table 10.2.1	Unit Testing Results	48
Table 10.2.2	Integration Testing	49
Table 10.2.3	System Testing	49
Table 10.2.4	User Acceptance Testing(UAT)	50
Table 10.2.5	Performance and Security Testing Results	50

IV. ACRONYMS

Acronym	Full Form
AI	Artificial Intelligence
ML	Machine Learning
CNN	Convolutional Neural Network
LSTM	Long Short-Term Memory
API	Application Programming Interface
IoT	Internet of Things
NDVI	Normalized Difference Vegetation Index
RGB	Red Green Blue
FAO	Food and Agriculture Organization
ICAR	Indian Council of Agricultural Research
IoU	Intersection over Union
KLOC	Kilo Lines of Code
EAF	Effort Adjustment Factor
UAT	User Acceptance Testing
UML	Unified Modeling Language
ER	Entity Relationship
DFD	Data Flow Diagram
UI	User Interface
GPU	Graphics Processing Unit
CPU	Central Processing Unit
JSON	JavaScript Object Notation

CHAPTER 1

INTRODUCTION

1.1 INTRODUCTION

Modern agriculture faces a complex set of challenges: changing weather patterns, increasing pest and disease pressure, inefficient use of water and fertilizers, and the need to achieve higher yield on limited land. Smallholder and commercial farmers alike often rely on experience and reactive decisions rather than continuous, data-driven guidance. At the same time, advances in artificial intelligence, computer vision, time-series modelling, and lightweight cloud infrastructure make it possible to deliver actionable intelligence to farmers in near real-time.

UVION is conceived as an AI-powered, multi-agent autonomous farming system that integrates several complementary AI capabilities into a single decision-support platform. Rather than addressing a single problem (for example, disease detection or irrigation scheduling), **UVION** brings together five tightly integrated agents—Growth Prediction, Disease Detection, Weather & Irrigation, Fertilizer Recommendation (RAG), and Yield Prediction—so the system can reason holistically about crop health and recommend prioritized, explainable actions. The system is designed to be demonstrable with zero hardware cost (via simulated sensors) while remaining ready to accept real IoT inputs in future deployments.

1.2 AIM & OBJECTIVES

Aim-

To design and implement **UVION**, an AI-based, multi-agent autonomous farming assistant that provides farmers with accurate growth forecasts, image-based disease diagnosis, weather-aware irrigation recommendations, scientifically grounded fertilizer plans, and reliable yield estimates—delivered via an intuitive dashboard and explainable advice.

Objectives-

- a. **Growth Forecasting:** Develop a time-series model to predict crop growth (e.g., growth index or proxy like NDVI/height) for the next 30 days, using historical data, virtual sensor streams, and weather forecasts.
- b. **Disease Detection:** Build a computer-vision module capable of classifying common crop diseases from leaf images and estimating disease severity (affected area).
- c. **Weather & Irrigation Planning:** Integrate weather forecast data and a soil-moisture model to recommend optimal irrigation timing and quantity.

- d. **Fertilizer Recommendation (RAG):** Implement a Retrieval-Augmented Generation (RAG) knowledge pipeline that retrieves authoritative agronomy guidelines and produces crop-stage specific fertilizer recommendations with clear explanations.
- e. **Yield Prediction:** Fuse multimodal outputs (growth forecast, disease impact, irrigation/fertilizer actions, weather) into a yield prediction model that reports expected production and uncertainty.
- f. **Explainability & Usability:** Provide transparent recommendations backed by retrieved references and easy visualizations on a React dashboard.
- g. **Zero-Cost Prototype:** Demonstrate a full end-to-end prototype using free computational resources and simulated sensors, enabling reproducibility without hardware purchase.

1.3 REASON FOR THE PROJECT

Agriculture is critical for food security and livelihoods worldwide. Improving decision quality at the farm level directly impacts yield, income, and resource sustainability. Existing tools tend to be fragmented: many products focus on either sensing, disease classification, or simple irrigation logic, but rarely on combining these signals to reason about cause-and-effect at field scale. UVION addresses this gap by: Bringing together multiple AI capabilities so that recommendations account for interactions (e.g., disease severity reduces growth potential and changes fertilizer needs).

In short, UVION is motivated by the need for an integrated, affordable, and explainable farm intelligence system that helps farmers make proactive, data-driven decisions.

First, import images with an optical device like a scanner or a camera or directly through digital processing.

Second, manipulate or analyse the images in some way. This step can include image improvement and data summary, or the images are analysed to find rules that aren't seen by the human eyes. For example, meteorologists use this processing to analyse satellite photographs.

Last, output the result of image processing. The result might be the image changed by some way or it might be a report based on analysis or result of the images.

1.4 PROBLEM STATEMENT

Small and large farms lose significant crop potential because decisions about irrigation, nutrient application, and pest/disease control are often made reactively or based on incomplete information. Key problems include: Untimely interventions, Poor nutrient timing and dosing,

Delayed disease detection.

UVION addresses these by providing an integrated, agentic AI system that forecasts growth, diagnoses diseases from images, recommends irrigation and fertilizer schedules based on weather and soil status, and predicts yield—helping farmers to act proactively and efficiently.

1.5 SCOPE

The scope of the **UVION** major project is deliberately focused to ensure a complete, demonstrable, and well-evaluated system within the constraints of an academic project while providing a modular foundation for future extension.

Implement the five agents (Growth, Disease, Weather/Irrigation, Fertilizer/RAG, Yield) as modular components. Use publicly available datasets (e.g., PlantVillage for disease images, open weather APIs, public yield/time-series datasets) and simulated sensor data to train and evaluate models. Build a FastAPI backend that orchestrates agents and a React frontend dashboard for interaction and visualization.

Provide clear explainability for fertilizer recommendations via RAG retrievals and present evidence citations. Demonstrate a complete end-to-end workflow: image upload → agent analysis → combined advisory → dashboard output.

Keep implementation cost at zero by using free compute (Google Colab/Kaggle), free-tier hosting, and simulated sensors.

1.6 SUMMARY

This chapter introduced **UVION**, its motivation, goals, and the exact problems it intends to solve. The project aims to construct an integrated, explainable, zero-cost prototype that merges growth forecasting, image-based disease detection, weather-aware irrigation planning, RAG-backed fertilizer advice, and multimodal yield prediction.

CHAPTER 2

LITERATURE SURVEY

2.1 SURVEY OF RELATED WORK

This survey is conducted based on existing research in crop disease detection, crop yield prediction, and smart agriculture using Machine Learning, Deep Learning, and IoT technologies. The reviewed studies focus on image-based disease identification, web-based prediction systems, and sustainable farming approaches.

SNO	Title of paper	Methodology	Results	Advantages	Drawbacks
1	"A Survey on Crop Disease Detection Using Image Processing Technique", 2020	Histogram Equalization for contrast enhancement, K-means clustering for classification, Image Processing for disease detection.	Detects diseases in crop leaves accurately using image processing techniques	Helps farmers prevent heavy crop losses, Automated system for early disease detection	Limited to leaf-based diseases, no real-time implementation discussed
2	"Deep Learning-Based Crop Disease Prediction with Web Applications (2023)"	This research implements convolutional neural networks (CNNs) for automated feature extraction and classification of plant diseases.	High accuracy with real-time disease prediction.	User-friendly web interface. Real-time prediction capability.	Requires high computational resources Dataset dependency.
3	"Machine Learning and Deep Learning for Crop Disease Diagnosis in Sustainable Agriculture (2024)"	Used deep learning algorithms, including decision trees, random forests, support vector machines, focusing on their effectiveness in sustainable agricultural disease diagnosis.	Classification accuracy between 83%-94% for five diseases.	High predictive accuracy, scalability, supports sustainable agriculture, adaptable to multiple crops.	Complex model architecture, high training time, requires extensive datasets, computationally expensive.
4	"Plant Disease Detection Using Machine Learning", 2018	Collecting leaf images, extracting features using the Histogram of Oriented Gradients (HOG), and training a Random Forest classifier to distinguish between	The approach showed promise in accurately distinguishing between healthy and diseased plants.	The combination of HOG for feature extraction and Random Forest for classification provides a balance	The Random Forest classifier, while robust, might not capture complex patterns as effectively as deep learning Models.

SNO	Title of paper	Methodology	Results	Advantages	Drawbacks
		healthy and diseased plants.		between computational efficiency and accuracy.	
5	"A Review on Crop Yield Prediction Using Machine Learning Methods (2021)"	This paper reviews multiple machine learning techniques such as linear regression, decision trees, random forests, and ensemble methods for predicting crop yield based on environmental and historical data.	The study concludes that ensemble models provide better prediction accuracy compared to individual.	Improves decision-making, handles multiple input parameters, better prediction with ensemble methods.	Dependent on data quality, limited real-time capability, requires preprocessing, less effective with missing data.
6	"Deep Learning-Based Crop Disease Prediction with Web Applications (2023)."	CNN-based architecture integrated with web technologies, focusing on real-time disease detection and user accessibility through browser-based platforms.	Reliable performance in disease prediction and enhances usability through an interactive interface.	Real-time prediction, accessible platform, high accuracy, practical.	Requires internet connectivity, computational overhead, dependency on dataset quality.
7	"A Survey on Deep Learning Based Crop Yield Prediction (2024)"	This research analyzes deep learning models such as CNN, RNN, and LSTM for capturing spatial and temporal patterns in agricultural data to predict crop yield.	Improved prediction accuracy over traditional models, especially when handling time-series and large-scale datasets.	Handles complex data patterns, high prediction accuracy, suitable for large datasets, captures temporal dependencies.	High computational cost, complex training process, requires large datasets, difficult model tuning.
8	" Smart Farming with IoT and Machine Learning (2023)"	This system integrates IoT sensors for real-time data collection (soil moisture, temperature, humidity) with machine learning models for crop recommendation and disease detection.	The approach enables real-time monitoring and improves farming decisions through automated insights.	Real-time monitoring, automation, improved decision-making, efficient resource utilization.	High implementation cost, dependency on network connectivity, maintenance of IoT devices, data security concerns.

SNO	Title of paper	Methodology	Results	Advantages	Drawbacks
9	"Recent Advances in Plant Disease Detection (2025)"	This paper explores advanced deep learning models combined with hyperspectral and RGB imaging techniques to enhance disease detection accuracy at early stages.	The study reports very high accuracy and early detection capabilities, significantly improving disease management in crops.	High precision, early detection, advanced imaging techniques, improved accuracy	Expensive equipment, complex data processing, high computational requirements, limited accessibility
10	"Crop Yield Prediction Using Machine Learning: Systematic Review (2024)"	Presents an integrated ML solution combining soil, weather and disease indicators to recommend crops and predict yields.	Demonstrates improved decision accuracy when combining disease detection outputs into yield.	Conceptual blueprint for UVION's multi-agent fusion — empirical evidence that integration improves overall utility.	Data collection and label alignment across modules remain a bottleneck.

2.2 BENEFITS OF THE PROPOSED PROJECT

The UVION system provides a comprehensive and intelligent solution for modern agriculture by integrating plant disease detection, fertilizer recommendation, and crop yield prediction into a single platform. This integration allows farmers to make accurate and timely decisions, thereby improving overall agricultural productivity and sustainability. The system enables early detection of plant diseases through image-based analysis, which helps in identifying infections at an initial stage. This reduces the risk of severe crop damage and minimizes losses that usually occur due to delayed diagnosis. By automating detection, the dependency on agricultural experts is also reduced.

- a. UVION improves crop productivity by recommending suitable fertilizers based on soil nutrient composition and environmental conditions. By ensuring that crops receive the right nutrients in the correct quantities, the system enhances plant growth and leads to better yield quality and quantity.
- b. The project promotes data-driven decision-making by analyzing multiple parameters such as soil nutrients, temperature, humidity, and rainfall. Instead of relying on traditional guesswork, farmers can make scientifically backed decisions that improve efficiency and reliability.

CHAPTER 3

EXISTING SYSTEM

3.1 INTRODUCTION

Over the past decade many technology solutions for farming have appeared. They target different parts of the farming problem: detecting plant diseases from pictures, suggesting when to water, recommending fertilizers from soil tests, predicting yields from remote-sensing data, and offering farm record keeping. However, most of these systems solve one specific problem well, not the whole lifecycle. In practice a farmer juggles many interdependent decisions — irrigation, nutrition, disease management — and the decisions interact (e.g., over-watering can worsen disease; missed fertilizer windows reduce growth). Existing platforms rarely combine these signals into a single, prioritized, explainable action plan that adapts to real-time observations.

Below we describe what typical existing systems do, how they work (high level), the common limitations, and the real problem statement UVION addresses.

What existing systems actually do — a detailed view

3.1.1 Disease-detection

Disease detection applications based on computer vision are widely used tools that help farmers identify plant diseases through image analysis. These systems allow farmers to capture or upload images of plant leaves, fruits, or other parts, which are then processed using trained Convolutional Neural Network (CNN) models. The models classify the images into categories such as specific diseases like tomato early blight or potato late blight, or identify them as healthy. In many cases, these applications also provide basic remedial suggestions, including pesticide usage or sanitation practices.

Internally, these systems operate by taking a single image as input, followed by preprocessing steps such as resizing, normalization, and sometimes background removal. The processed image is then passed through a CNN backbone like ResNet or EfficientNet along with a classification layer. The output generated includes the predicted disease label along with a confidence score, and in some advanced implementations, visual aids such as heatmaps or bounding boxes are provided to highlight affected regions. From a farmer's perspective, the results typically include the disease name, a confidence level, an optional urgency indicator, and a short treatment recommendation or reference link.

However, these systems have several limitations. Although they perform well on curated datasets, their accuracy often decreases in real-world conditions due to factors like poor

lighting, overlapping leaves, or dust. They generally do not estimate disease severity or track progression over time, and they operate independently without integrating with other agricultural decisions such as irrigation or fertilizer planning.

Irrigation controllers and smart scheduling systems are another category of agricultural technologies designed to optimize water usage. These systems provide irrigation schedules either through predefined rules or basic machine learning models. Some advanced systems use real-time soil moisture sensors and weather forecasts to determine irrigation needs, while simpler systems function as timers with optional remote control through mobile applications.

These systems operate by collecting input data such as soil moisture levels or weather forecasts. Based on this data, rule-based logic or simple regression models determine irrigation actions. For example, if soil moisture falls below a certain threshold, the system triggers irrigation. The output includes irrigation schedules specifying start time, duration, and sometimes automated control of pumps. Farmers typically receive notifications such as instructions to irrigate at a specific time or are given options to manually control irrigation through an application. Despite their usefulness, these systems also have limitations. The thresholds used are often static and do not adapt to changing factors such as crop growth stage, disease conditions, or recent fertilizer applications. Many systems require hardware installation, which increases cost and limits accessibility. Additionally, they lack predictive capabilities and do not effectively coordinate irrigation decisions with weather conditions, such as avoiding watering before rainfall.

3.1.2 Fertilizer recommendation tools

Fertilizer recommendation tools and expert systems assist farmers in determining the appropriate type and quantity of fertilizers based on soil and crop conditions. These systems take inputs such as soil nutrient levels, including nitrogen, phosphorus, and potassium, along with pH value, crop type, and growth stage. Based on this information, they recommend suitable fertilizers, dosage, and application timing. Most traditional systems operate using rule-based approaches, where predefined if-then rules are applied. For instance, if nitrogen levels are below a certain threshold and the crop is in a vegetative stage, a specific quantity of urea may be recommended. More recent systems incorporate retrieval-based approaches or decision tree models to enhance recommendations. The output provided to farmers typically includes the fertilizer name, the required amount per hectare, and the recommended application schedule.

3.2 KEY CHALLENGES

The following are the key challenges of the UVION system:

a. Siloed operation

Different tools treat problems separately; no single platform fuses disease, irrigation, fertilizer, and growth forecasts into a prioritized plan.

b. Domain and data mismatch

Models trained on clean/lab datasets often fail on noisy field images. Satellite-derived indices may not capture field-scale variability.

c. Data scarcity & annotation cost

High-quality labeled datasets (disease, yield ground-truth) are expensive. Many crops/locales lack representative data.

d. Dependency on hardware

Systems requiring physical sensors face adoption barriers: cost, installation, maintenance, and calibration.

e. Opaque recommendations

Many systems are “black box” — they give prescriptions without showing the evidence, origins, or confidence. Farmers distrust such outputs.

3.3 PROBLEM STATEMENT

Agriculture remains a primary source of livelihood for a large population, yet it continues to face persistent challenges due to the lack of timely information, limited access to expert guidance, and inefficient resource management. Farmers often rely on traditional methods and personal experience to identify plant diseases, select fertilizers, and estimate crop yield, which can lead to inaccurate decisions and reduced productivity. One of the major problems is the difficulty in accurately identifying plant diseases at an early stage. Visual inspection by farmers is often unreliable, especially when symptoms are similar across different diseases. This results in delayed treatment, leading to significant crop damage and financial losses. Existing solutions are often fragmented, addressing only one aspect such as disease detection or fertilizer recommendation.

CHAPTER 4

PROPOSED SOLUTION

4.1 INTRODUCTION

Based on the limitations identified in existing agricultural systems—such as siloed tools, hardware dependency, lack of explainability, poor integration, and high cost—the proposed solution is **UVION**, an AI-powered, multi-agent autonomous farming system. Unlike traditional approaches that treat each farm task separately (disease detection, irrigation, fertilizer advice, yield prediction), UVION integrates these components into a unified ecosystem of intelligent agents that communicate, collaborate, and reason across the entire crop lifecycle.

UVION combines machine learning, computer vision, retrieval-augmented generation (RAG), and simulated IoT sensor data to provide a comprehensive, data-driven farming decision assistant. It predicts crop growth, identifies diseases from images, recommends irrigation and fertilizer schedules, predicts yield, and prioritizes actions based on the overall farm condition. The system operates at zero hardware cost, thanks to simulated sensors, and uses only free-tier services and open-source tools.

UVION's goal is not only automation but intelligent decision orchestration—helping farmers understand what to do, when to do it, and why it should be done, backed by scientific explanations.

4.2 ADVANTAGES OF THE PROPOSED SYSTEM

The UVION system offers several improvements over existing tools. These advantages are both technical and practical for real-world farming.

a. Multi-Agent Integrated Decision-Making

The UVION system provides several important improvements over existing agricultural tools by combining both technical innovation and practical usability for real-world farming. One of its key advantages is the use of a multi-agent integrated decision-making approach, where different modules such as growth prediction, disease detection, weather and irrigation forecasting, fertilizer recommendation, and yield prediction work together instead of operating separately. This collaboration allows the system to generate a unified and more accurate advisory, unlike traditional siloed systems that provide fragmented suggestions.

b. Zero Hardware Cost Through Simulated Sensors

Physical IoT sensors increase cost and maintenance burden. UVION includes a virtual sensor engine that generates realistic soil moisture, temperature, humidity, and soil EC data.

c. Scientific and Explainable Recommendations (RAG System)

UVION also ensures scientific and explainable recommendations through its Retrieval-Augmented Generation (RAG) system. By retrieving relevant information from trusted agricultural sources such as FAO and ICAR documents, the system provides fertilizer recommendations along with clear explanations and supporting evidence. This enhances transparency and builds user trust in the system's outputs.

d. Predictive Instead of Reactive Farming

In addition, the system promotes predictive farming rather than reactive decision-making. Instead of only identifying current problems, UVION forecasts future growth patterns, irrigation needs, yield outcomes, and the potential impact of diseases and weather conditions. This allows farmers to take proactive actions, reducing risks and improving productivity.

4.3 SPECIFICATIONS OF THE PROPOSED SYSTEM

The proposed solution, UVION, is designed as an integrated intelligent agricultural system that combines deep learning and machine learning techniques to address key farming challenges. It provides a unified platform for plant disease detection, fertilizer recommendation, and crop yield prediction, ensuring efficient and data-driven agricultural practices.

4.3.1 Disease Detection Module

The system incorporates a Convolutional Neural Network (CNN) model to detect plant diseases using leaf images. The model is trained on large-scale datasets, including the extended PlantVillage Dataset, enabling it to classify plant conditions into healthy and multiple disease categories. The use of RGB image inputs ensures compatibility with real-world images captured through mobile devices, improving usability for farmers.

4.3.2 Fertilizer Recommendation Module

This module uses supervised machine learning algorithms to analyze soil and environmental parameters. It considers nutrient values such as nitrogen, phosphorus, and potassium along with factors like temperature, humidity, and soil type. Based on these inputs, the system recommends the most appropriate fertilizer, helping in maintaining soil fertility and improving crop growth.

4.3.3 Crop Yield Prediction Module

The yield prediction component estimates agricultural output using predictive models trained on historical and environmental data. By evaluating multiple parameters, including crop type and climatic conditions, the system provides an expected yield value, which assists farmers in planning and resource management.

4.3.4 Dataset Integration and Processing

The system utilizes multiple datasets sourced from platforms such as Kaggle along with real-world field data. This integration ensures better generalization and performance under diverse agricultural conditions. All datasets undergo preprocessing steps such as normalization, resizing, feature scaling, and data augmentation to improve model accuracy.

4.3.5 System Architecture

The architecture of UVION is modular in nature, allowing each component to operate independently as well as in an integrated manner. This design improves system flexibility, making it easier to update or expand individual modules without affecting the entire system.

The system is designed with a user-friendly interface that enables easy interaction. Farmers can upload images or input data and receive instant results in a simple and understandable format. The interface supports multilingual outputs, including English, Hindi, and Telugu, ensuring accessibility for users from different regions.

CHAPTER 5

SYSTEM ANALYSIS

5.1 INTRODUCTION

System analysis is a critical phase in the development of the UVION project, as it focuses on understanding the existing agricultural challenges, identifying system requirements, and defining the overall structure of the proposed solution. This phase ensures that the system is designed efficiently to meet user needs while addressing real-world farming problems through technology. The agricultural sector faces multiple issues such as inaccurate disease identification, improper fertilizer usage, and uncertainty in crop yield prediction. Traditional methods rely heavily on manual observation and experience, which are often inconsistent and prone to error. These limitations highlight the need for a smart, automated system that can provide reliable and timely insights to farmers.

In the context of UVION, system analysis involves studying how different components such as plant disease detection, fertilizer recommendation, and yield prediction can be integrated into a single platform. It also includes analyzing user requirements, data sources, processing techniques, and system outputs to ensure seamless functionality. The goal is to create a system that is not only accurate but also user-friendly and accessible to individuals with minimal technical knowledge.

5.2 FEASIBILITY STUDY

The feasibility study is an essential phase in the development of the UVION system, as it evaluates whether the proposed solution is practical, viable, and beneficial for real-world implementation. This phase focuses on analyzing different aspects of the system, including technical requirements, economic considerations, operational usability, and overall effectiveness in addressing agricultural challenges.

In the context of UVION, the feasibility study examines how well the integration of plant disease detection, fertilizer recommendation, and crop yield prediction can be implemented using current technologies. It assesses whether the required resources such as datasets, computational tools, and development frameworks are available and sufficient to build an efficient system. The study also considers the adaptability of the system in diverse agricultural environments, ensuring that it can function effectively under varying conditions.

5.2.1 Technical Feasibility

Technical feasibility evaluates whether the UVION system can be successfully developed using available technologies, tools, and resources. The project relies on well-established techniques from Machine Learning and Deep Learning, which are widely used for solving real-world problems involving image classification and predictive analytics.

The disease detection module is implemented using Convolutional Neural Networks, which are highly effective for processing image data. Training such models is feasible due to the availability of large datasets like the PlantVillage Dataset and other extended datasets. The fertilizer recommendation and yield prediction modules are based on supervised learning algorithms, which can efficiently handle structured agricultural data.

5.2.2 Operational Feasibility

Operational feasibility focuses on how effectively the UVION system can be used in real-world agricultural environments. The system is designed to be user-friendly and accessible, ensuring that farmers can easily interact with it without requiring advanced technical knowledge.

The application allows users to upload plant images or input soil and environmental data to receive instant recommendations. The interface is kept simple and intuitive, enabling smooth operation even for first-time users. Additionally, multilingual support ensures that users from different regions can understand the system outputs.

5.2.3 Economical Feasibility

Economic feasibility determines whether the UVION project is cost-effective and financially viable. The system is designed to minimize development and operational costs while maximizing benefits for farmers.

The use of open-source tools and platforms significantly reduces development expenses. Cloud-based services eliminate the need for high-end infrastructure, making the system affordable to build and maintain. Moreover, the benefits provided by the system outweigh the costs involved.

Overall, the UVION system is economically feasible as it delivers high value at a relatively low cost, making it a practical solution for modern agriculture.

5.2.4 COCOMO MODEL ESTIMATION

The COCOMO model estimates the effort, development time, and cost of a software project based on the size of the code measured in KLOC (Kilo Lines of Code). Since UVION is a moderately complex system involving machine learning models, APIs, and a web interface, it is categorized under the Semi-Detached Mode.

The UVION project falls under the Semi-Detached category because it involves moderate complexity, integration of multiple modules such as CNN-based disease detection, machine learning-based prediction systems, and a user interface, with a team having mixed experience levels.

Basic COCOMO Model Calculation

The effort estimation is calculated using the formula:

$$E = a \times (KLOC)^b$$

For Semi-Detached projects:

$$a = 3.0$$

$$b = 1.12$$

Assuming the total size of the UVION system is approximately:

$$LOC = 4000$$

$$KLOC = 4$$

Now,

$$E = 3.0 \times (4)^{1.12}$$

$$E \approx 3.0 \times 4.72 = 14.6 \text{ person-months}$$

Development Time Estimation

The development time is calculated using:

$$D = 2.5 \times (E)^{0.35}$$

$$D = 2.5 \times (14.6)^{0.35}$$

$$D \approx 2.5 \times 2.5 = 6 \text{ months}$$

Team Size Estimation

$$N = \frac{E}{D}$$

$$N = \frac{14.6}{6} \approx 2 \text{ persons}$$

Thus, approximately 4 team members are required to complete the project within the estimated duration.

Intermediate COCOMO Model

The intermediate model considers additional cost drivers using the Effort Adjustment Factor (EAF).

$$E = a \times (KLOC)^b \times EAF$$

Assuming:

EAF = 1.2 (considering moderate complexity, dataset handling, and integration)

$$E = 3.0 \times (4)^{1.12} \times 1.2$$

$$E = 3.0 \times 4.72 = 16.2 \text{ person-months}$$

Now,

$$D = 2.5 \times (16.2)^{0.35} \approx 2.5 \times 2.7 = 6.7 \approx 7 \text{ months}$$

$$N = \frac{16.2}{7} \approx 3 \text{ persons}$$

Cost Estimation

Assuming the cost per person per month is ₹1,000:

$$\text{Total Cost} = E \times \text{Cost per person-month}$$

$$\text{Total Cost} = 16.2 \times 1000$$

$$\text{Total Cost} \approx ₹16200$$

5.2.5 COCOMO II Model

The COCOMO II model is an advanced version of the original COCOMO model, developed to provide more accurate cost estimation for modern software systems. Unlike the basic model, which relies only on lines of code, COCOMO II considers multiple factors such as system complexity, reuse, development environment, and team capability. This makes it more suitable for projects like UVION, which involve machine learning models, web development, and integration of multiple modules.

In the UVION project, the COCOMO II model is applied using the **Object Point approach**, as the system includes user interfaces, data processing modules, and API-based interactions rather than purely traditional coding structures.

$$NOP = \text{Object Points} \times \frac{(100 - \%reuse)}{100}$$

$$NOP = 38 \times \frac{(100 - 10)}{100}$$

$$NOP = 38 \times 0.9 = 34.2 \approx 34$$

Productivity Rate

The productivity rate depends on the developer experience and environment. For UVION, since the project uses standard tools, pre-trained models, and moderate complexity, the productivity is considered **Medium**.

Productivity Rate = 6 object points per person-month

Effort Estimation

Effort is calculated as:

$$Effort = \frac{NOP}{Productivity\ Rate}$$
$$Effort = \frac{34}{6} \approx 5.66 \approx 6\ person-months$$

Development Time Estimation

The development time depends on team size and effort distribution. Assuming a team of 3 members:

$$Duration = \frac{Effort}{Team\ Size}$$
$$Duration = \frac{6}{3} = 2\ months$$

Cost Estimation

Assuming the cost per person per month is ₹1,000:

$$Total\ Cost = Effort \times Cost\ per\ person-month$$
$$Total\ Cost = 6 \times 1000 = ₹6,000$$

The COCOMO II estimation indicates that the UVION system can be developed with moderate effort and cost due to the use of modern tools, reusable components, and pre-trained models. Compared to traditional development, the effort is reduced because machine learning frameworks and APIs accelerate development.

5.2.6 Legal Feasibility

Legal feasibility evaluates whether the UVION system complies with applicable laws, regulations, and ethical standards related to software development, data usage, and deployment. This analysis ensures that the system can be implemented without violating legal requirements and that it operates within a safe and responsible framework.

The UVION project primarily uses publicly available datasets, including those sourced from platforms such as Kaggle and research datasets like the PlantVillage Dataset. These datasets are generally released for academic and research purposes, which makes their usage legally permissible, provided that proper attribution is given and licensing terms are followed. Care is taken to ensure that no copyrighted or restricted data is used without authorization.

5.3 SYSTEM IMPLEMENTATION

The system implementation phase of UVION focuses on transforming the designed architecture into a fully functional application by integrating all modules, including plant disease detection, fertilizer recommendation, and crop yield prediction. This phase involves the development of backend services, model training, frontend interface design, and deployment of the system in a user-accessible environment.

The UVION system is implemented using a modular approach, where each component is developed independently and later integrated into a unified platform. The implementation begins with data collection and preprocessing, followed by model development, API creation, and user interface integration. This structured approach ensures flexibility, scalability, and ease of maintenance.

The disease detection module is implemented using Convolutional Neural Networks trained on large-scale datasets such as the PlantVillage Dataset. The model is developed using deep learning frameworks and trained to classify plant leaf images into healthy or diseased categories.

The fertilizer recommendation module is implemented using supervised machine learning algorithms trained on structured agricultural datasets. The crop yield prediction module uses predictive modeling techniques to estimate agricultural output.

5.4 FUNCTIONAL REQUIREMENTS

Functional requirements describe what the system must do.

Disease Detection- Accepts crop leaf image & Classifies disease and estimates severity

Growth Prediction- Uses time-series data to forecast 30-day crop growth

Weather & Irrigation Agent- Fetches weather data & Suggests irrigation timing and quantity

Fertilizer Recommendation (RAG)- Retrieves agricultural knowledge, recommends fertilizer dose and timing & provides explanation for recommendations

Yield Prediction- Estimates final yield using multimodal inputs

Virtual Sensor Engine- Simulates soil/environment values

Dashboard Interface- Displays predictions, graphs, and advisories

Orchestrator- Combines agent outputs and prioritizes actions

5.5 NON-FUNCTIONAL REQUIREMENTS

Non-functional requirements define **system quality attributes**.

Performance- Fast inference response (<3 seconds for predictions)

Reliability- Agents operate independently; failure of one agent does not break the system.

Scalability- Modular design enables future addition of more crops and more agents.

Security- Secure endpoints, safe image handling, no personal data stored.

Usability- Simple dashboard navigation

5.6 HARDWARE & SOFTWARE REQUIREMENTS

Hardware Requirements

Component	Requirement
System RAM	Minimum 4 GB
Processor	Intel i3/i5 or equivalent
Storage	5–10 GB free space

Table 5.6.1

Software Requirements

Software	Purpose
Python 3.10+	Backend, ML models
FastAPI	API development
PyTorch / Scikit-Learn	Model training & inference
React.js	Frontend dashboard
Node.js	Running frontend
Google Colab	Free GPU training
OpenWeatherMap API	Weather data

Table 5.6.2

CHAPTER 6

SYSTEM DESIGN

System design plays a crucial role in defining the structure, flow, and interactions of various components in the UVION-An AI powered Autonomous Farming System. This chapter outlines various UML diagrams that illustrate how data is processed and components interact.

6.1 SYSTEM ARCHITECTURE DESIGN

The System Architecture of UVION consists of three main layers: the frontend, backend, and processing modules. The frontend is a user interface dashboard where farmers interact with the system. The backend, built using FastAPI, acts as the central controller that manages data flow and communication between modules. The processing layer includes modules such as disease detection (CNN), yield prediction (ML), weather analysis (API), and fertilizer recommendation (rule-based system).

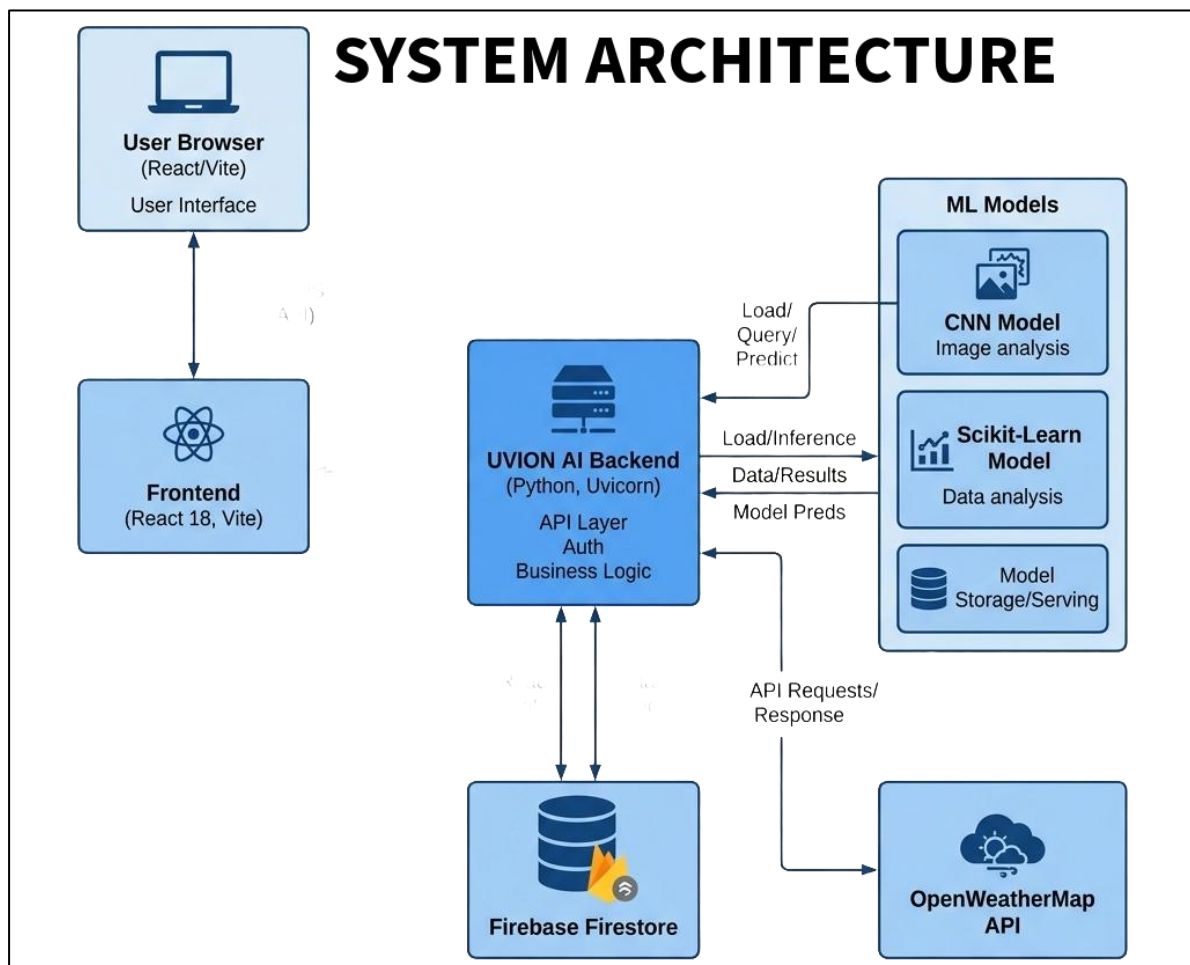


Fig 6.1 System Architecture

The system utilizes a virtual data generator to simulate environmental and soil conditions, eliminating the need for physical sensors. External weather APIs provide real-time environmental data such as temperature, humidity, and rainfall. The AI processing layer includes modules for disease detection, yield prediction, and growth estimation. The outputs from these modules are passed to the decision engine, which applies rule-based logic to combine all insights and generate actionable recommendations.

6.1.1 Data Flow Diagram

The Data Flow Diagram of UVION illustrates how data moves through the system. The primary external entity is the User, who provides inputs such as crop details and plant images. The system processes this input through multiple modules including growth analysis, disease detection, weather analysis, and yield prediction. Weather data is fetched from external APIs, while image data is processed using AI models.

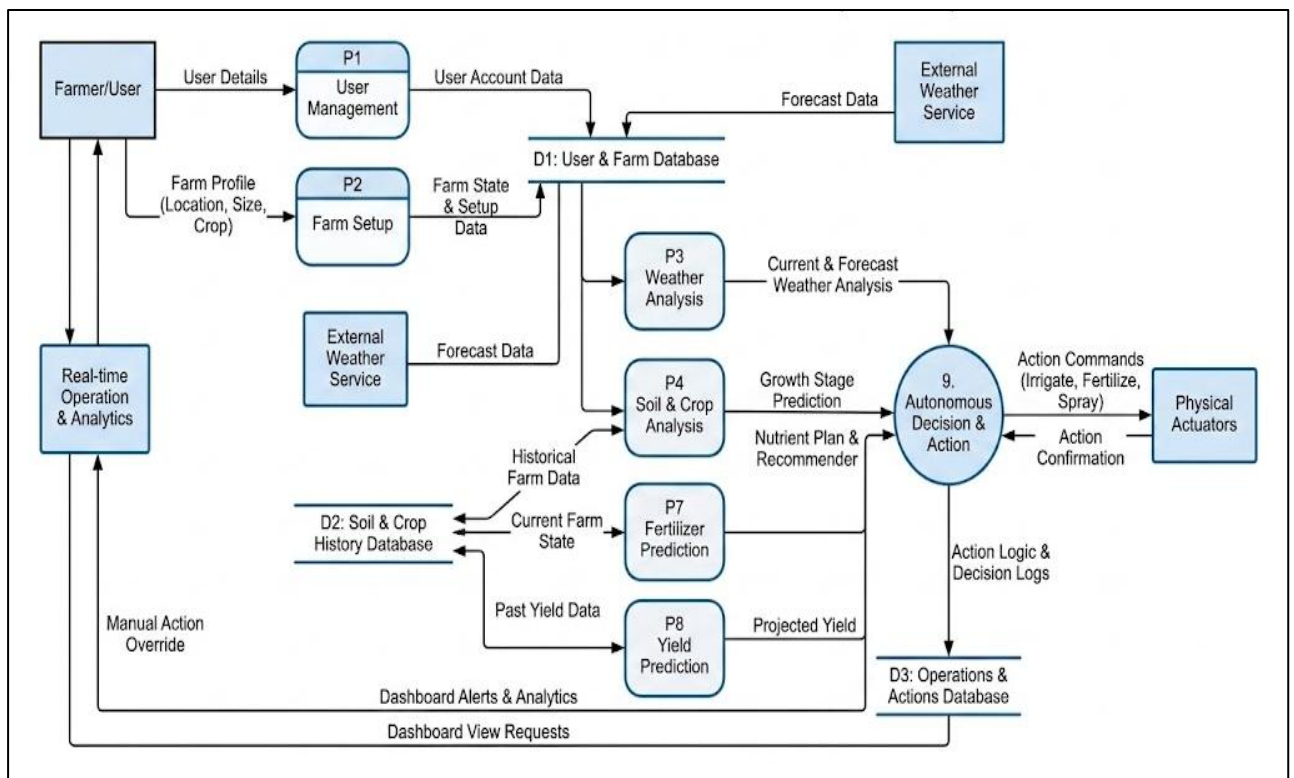


Fig 6.1.1 Data Flow Diagram

All processed data flows into the decision engine, which generates actionable recommendations. The final output, including predictions, alerts, and advice, is displayed on the dashboard.

6.1.2 E-R Diagram

The Entity-Relationship (E-R) Diagram of the UVION system represents the database structure and relationships between different entities. The main entities include User, Farm, Crop, CropSession, Image, Analysis, Weather, Prediction, and Recommendation.

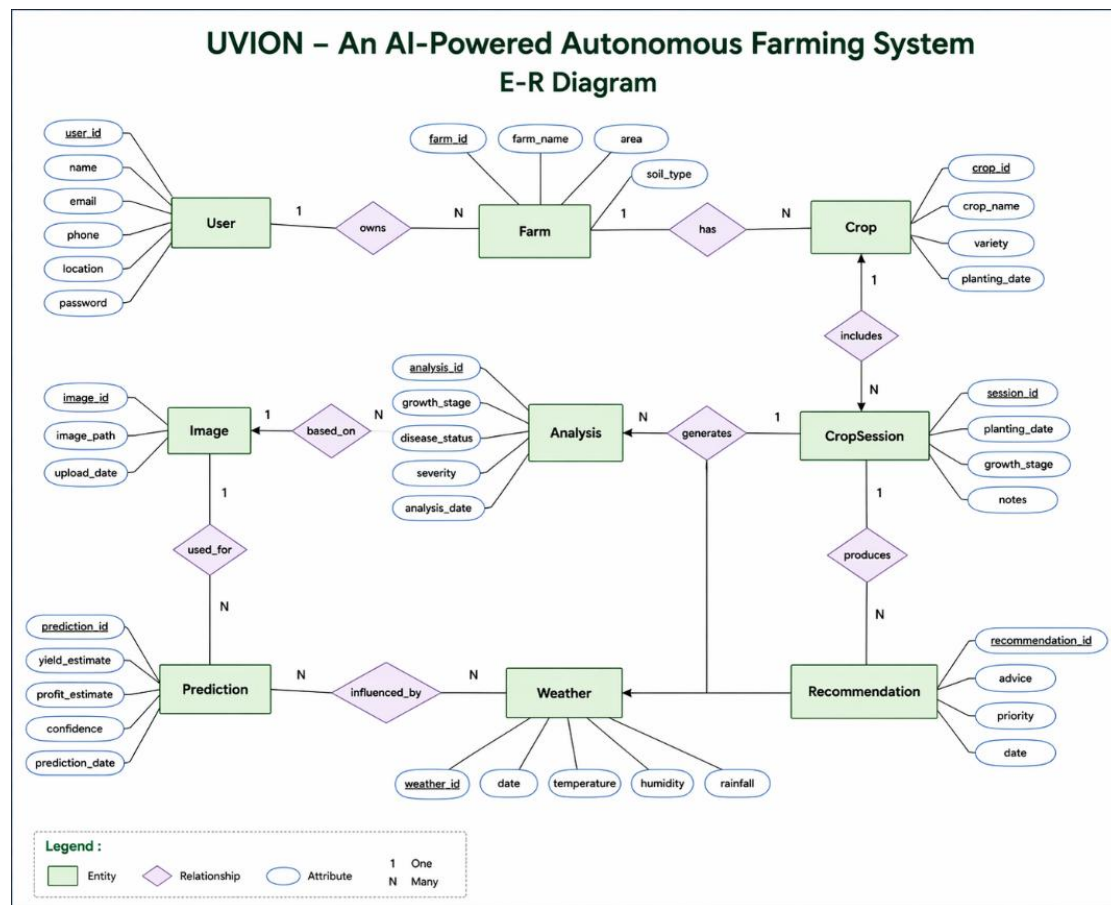


Fig 6.1.2 E-R Diagram

A User owns one or more farms, and each farm contains multiple crops. Each crop can have multiple crop sessions representing different cultivation cycles. Images uploaded by the user are linked to crops and are used for analysis. The Analysis entity processes crop data and generates insights such as disease status and growth stage.

The Prediction entity stores yield and profit estimations, which are influenced by weather conditions. The Recommendation entity provides actionable advice based on analysis and predictions. The relationships between these entities ensure efficient data organization and support the decision-making process of the system.

6.2 UML DIAGRAMS

Unified Modeling Language (UML) diagrams are used to visually represent the structure and behavior of the UVION system. These diagrams help in understanding how different components interact and how data flows within the system. In this project, UML diagrams are used to describe user interactions, system workflow, structural design, and component relationships.

6.2.1 Use Case Diagram

The Use Case Diagram represents the interaction between users and the UVION system. The primary actor is the Farmer, who interacts with the system through various functionalities such as registration, entering farm details, uploading plant images, and viewing predictions.

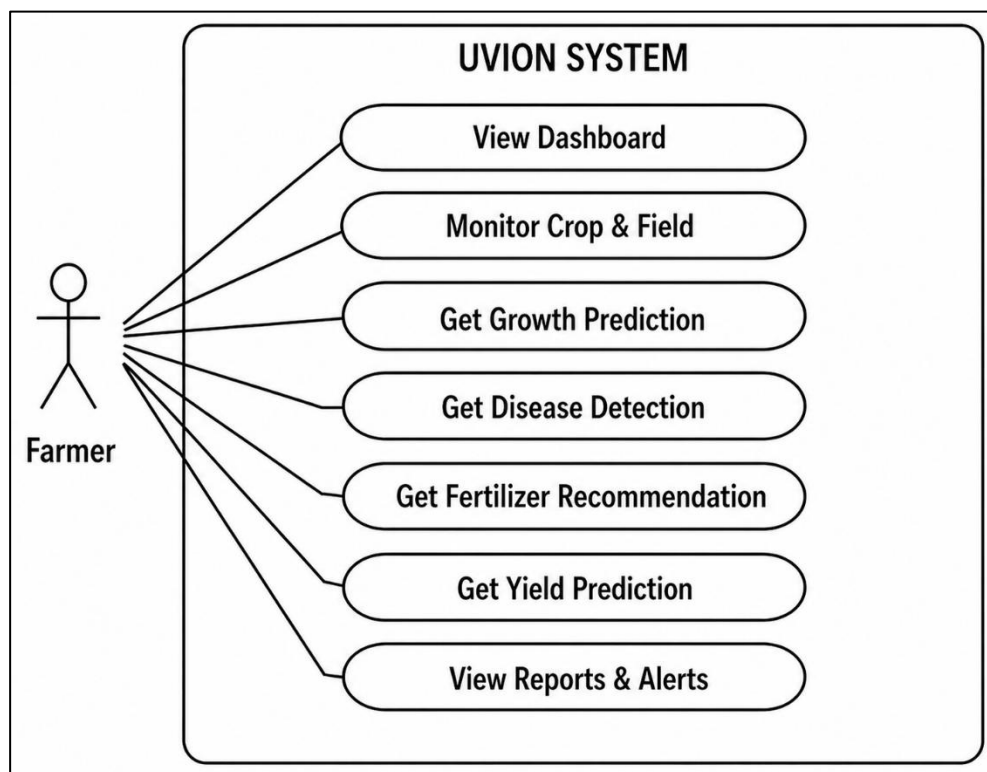


Fig 6.2.1 Use Case Diagram

The system provides multiple services including growth analysis, disease detection, weather insights, yield prediction, and final recommendations.

6.2.2 Sequence Diagram

The Sequence Diagram illustrates the step-by-step interaction between system components over time. The process begins when the user provides input through the frontend dashboard.

The request is sent to the backend, which processes the input and retrieves virtual environmental data.

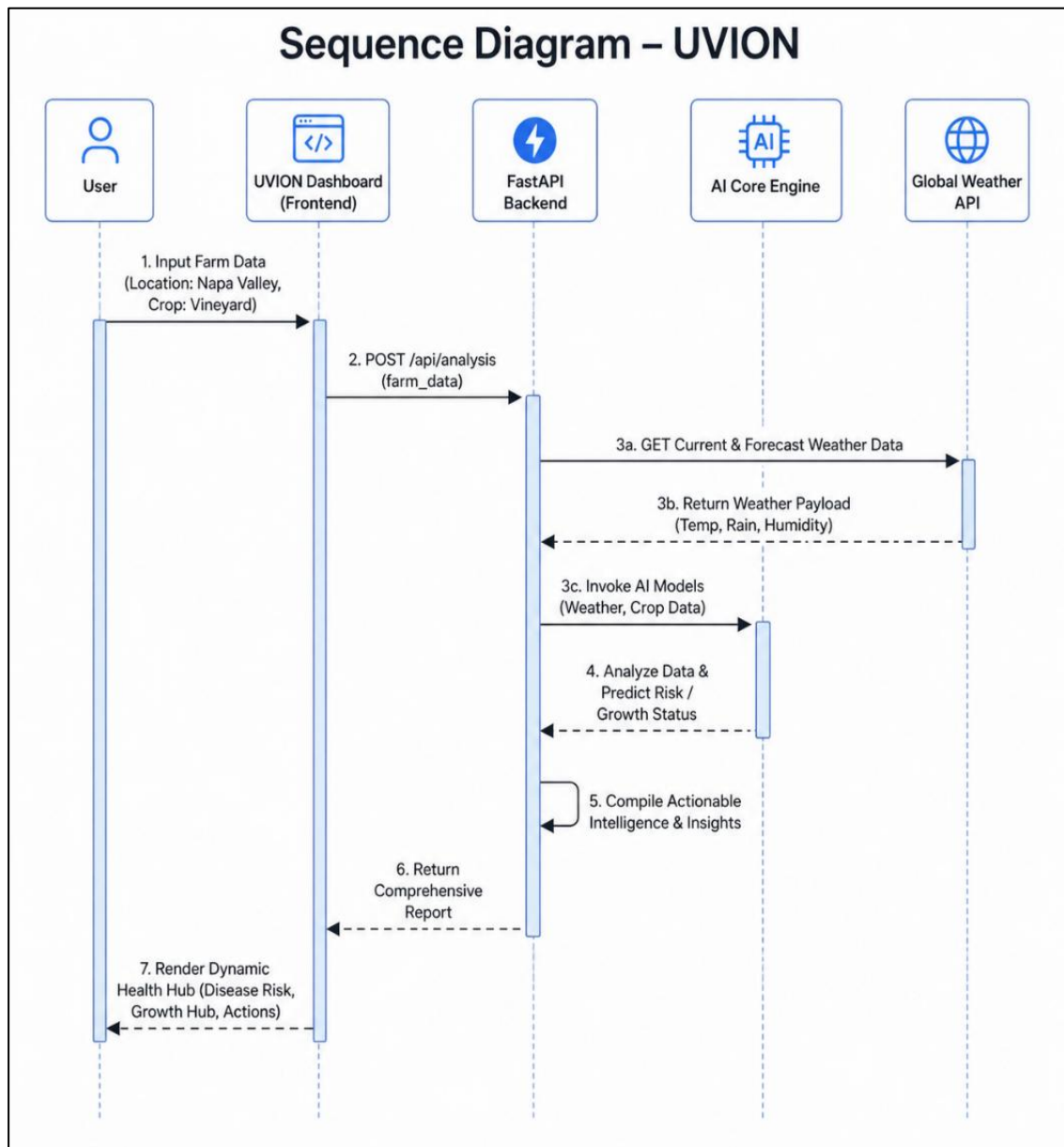


Fig 6.2.2 Sequence Diagram

The outputs from these modules are combined in the decision engine, which generates final recommendations. The results are then returned to the frontend and displayed to the user.

Once all computations and data integrations are complete, the backend constructs a structured response containing the detected disease, confidence levels, recommended actions, and associated insights. This response is sent back to the frontend, where it is displayed in a user-friendly format through dashboards, charts, and recommendation panels.

For modules such as fertilizer recommendation and yield prediction, a similar sequence is followed. The backend aggregates inputs from multiple sources, including crop type, growth stage, weather conditions, and previous actions, processes them using rule-based or predictive logic, and generates actionable outputs that are presented to the user.

Overall, the sequence diagram demonstrates a coordinated flow of information across system components, highlighting the real-time interaction between user inputs, intelligent processing modules, and data storage systems. This structured communication ensures that the UVION platform delivers accurate, timely, and actionable insights for crop management.

6.2.3 Class Diagram

The Class Diagram represents the structural design of the UVION system by defining the main classes and their relationships. The key classes include User, Farm, Crop, Image, Analysis, Weather, Prediction, and Recommendation.

The User class stores farmer details, while the Farm class represents farm-related information such as soil type and area.

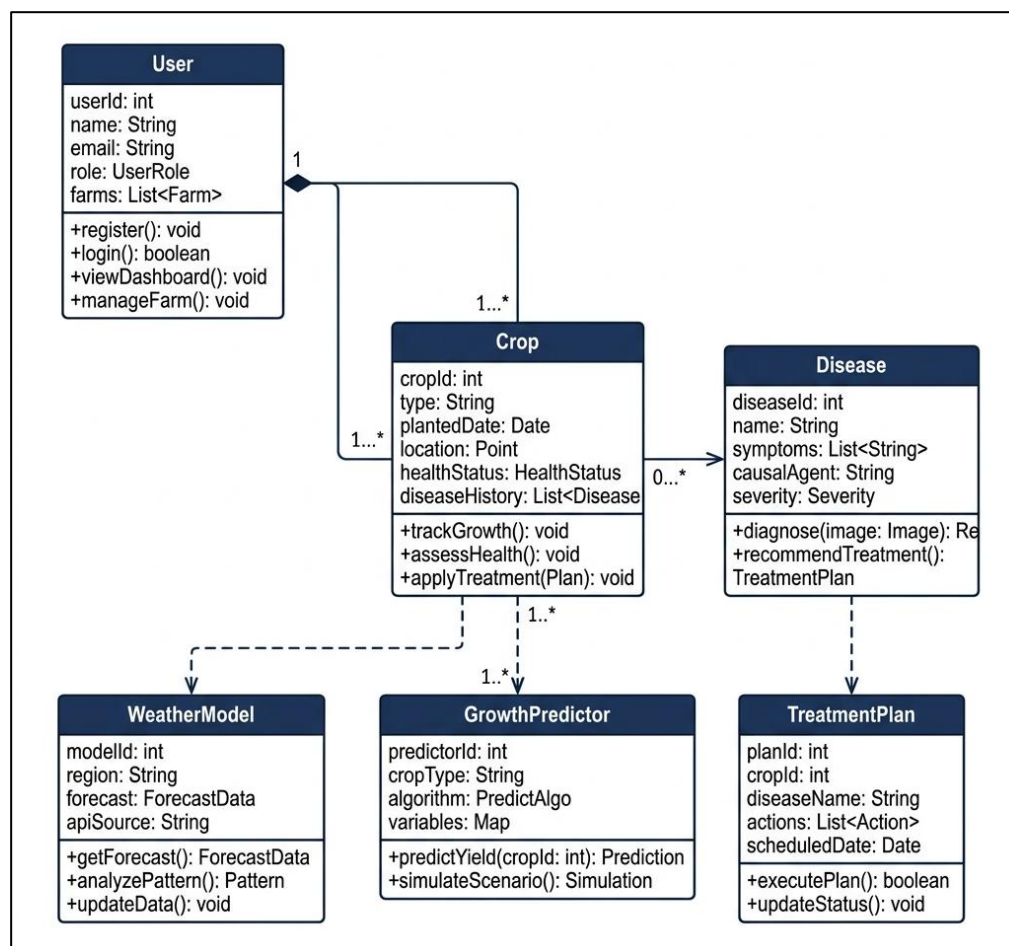


Fig 6.2.3 Class Diagram

This diagram helps in understanding how data and functionalities are organized within the system.

6.2.4 Activity Diagram

The Activity Diagram illustrates the workflow of the UVION system from input to output. The process begins with user authentication, followed by entering farm and crop details. The system then fetches virtual data such as weather, soil, and growth information.

If the user uploads a plant image, it is processed for disease detection. The collected data is passed to different modules including growth analysis, yield prediction, fertilizer recommendation, and weather analysis. The decision engine integrates all outputs and generates final recommendations.

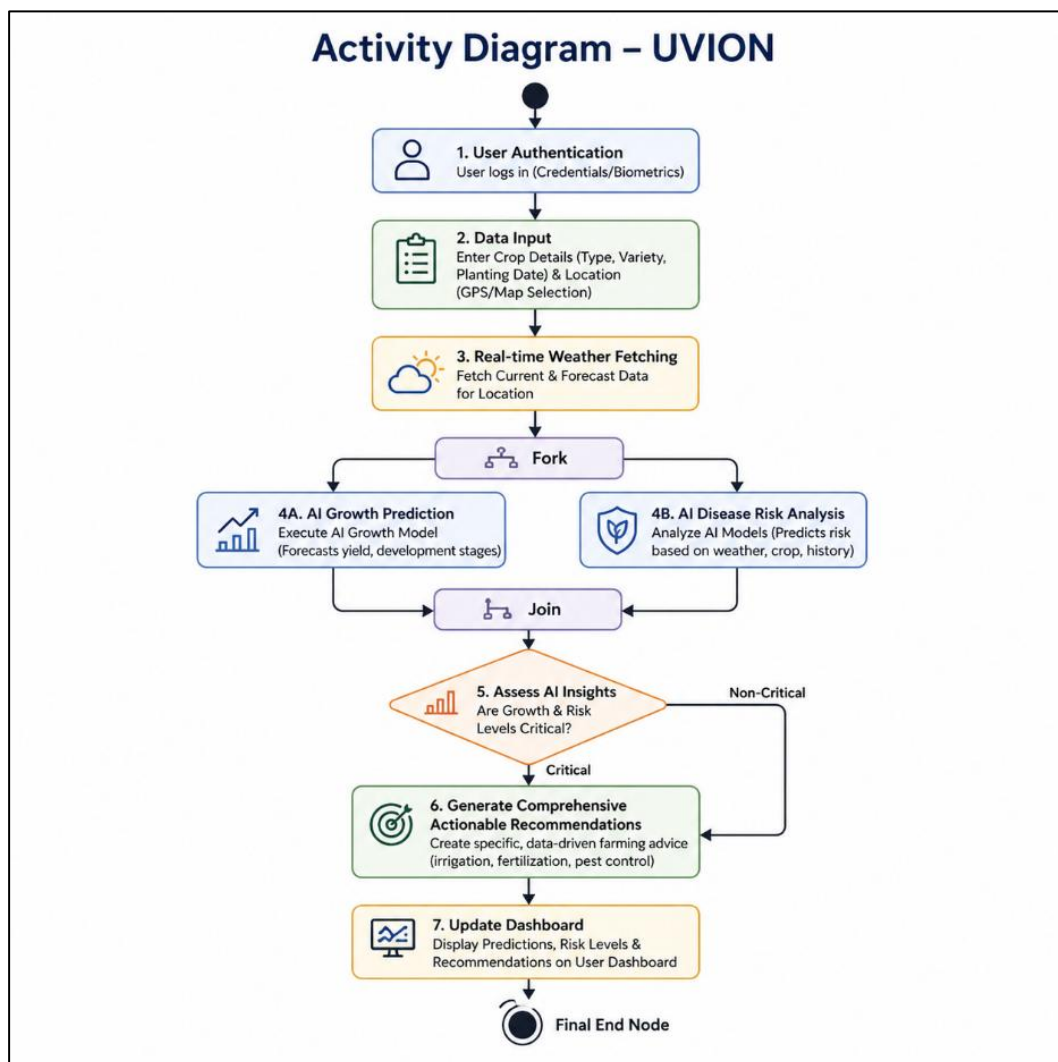


Fig 6.2.4 Activity Diagram

This diagram provides a clear representation of the system's operational flow.

CHAPTER 7

METHODOLOGY

The methodology of the UVION system focuses on designing an intelligent decision-support system that integrates multiple modules such as crop monitoring, disease detection, weather analysis, and yield prediction. The system follows a modular and step-by-step approach, where user inputs and virtual environmental data are processed to generate actionable recommendations. The methodology ensures simplicity, scalability, and efficient decision-making without relying on physical hardware.

7.1 System Approach

The **UVION** system follows a multi-stage processing approach, where data is collected, analyzed, and integrated to produce final recommendations. The system consists of the following stages:

The **UVION** system begins by collecting essential data from the user through the frontend dashboard. The user provides inputs such as crop type, planting date, farm location, optional soil type, and plant images for disease detection. This initial data acts as the foundation for all further processing and analysis within the system. After collecting inputs, the system generates and retrieves necessary environmental data. Instead of relying on physical sensors, UVION uses a virtual data generation approach to simulate conditions such as soil properties and growth parameters. Real-time weather data including temperature, humidity, and rainfall is obtained through APIs. This approach reduces hardware dependency and ensures cost-effective implementation.

Data Generation and Retrieval - Since the system does not rely on physical sensors, it uses a virtual data generation approach to simulate environmental conditions. The system processes the collected and generated data through multiple specialized modules. The disease detection module uses a Convolutional Neural Network to analyze plant images and identify disease type, severity, and treatment suggestions. The growth analysis module evaluates planting date and environmental conditions to determine crop growth stages. The weather analysis module interprets weather data to generate insights and alerts. The fertilizer recommendation module provides suggestions based on soil conditions and crop type using rule-based logic. The yield prediction module applies machine learning techniques to estimate crop yield and potential profit.

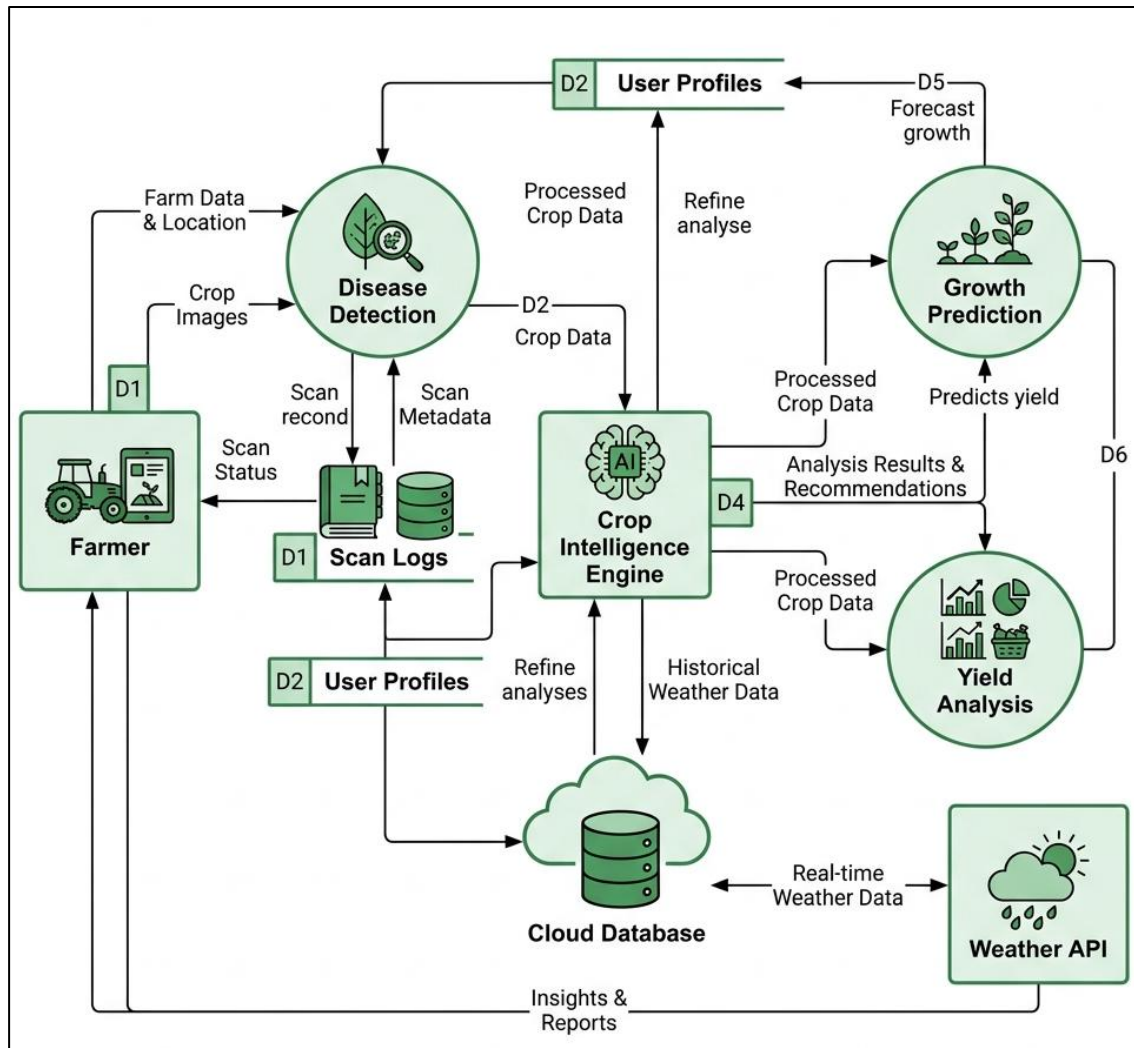


Fig 7.1 System Approach

Weather data can be obtained using APIs, while other parameters are simulated using predefined rules and datasets. This approach reduces cost and simplifies system implementation.

The decision engine acts as the core of the UVION system by integrating outputs from all modules. It applies rule-based logic to analyze different conditions and generate final recommendations. For example, if disease severity is high, it suggests immediate treatment, if humidity is high, it raises disease risk alerts, and if growth is slow it recommends fertilizer application. This ensures that all aspects are considered collectively for better decision-making.

The final stage involves presenting the results to the user through the dashboard. The system displays growth analysis, weather insights, disease detection results, yield and profit predictions, and actionable recommendations. The output is designed to be simple, clear, and easy to understand, enabling farmers to make informed decisions effectively.

7.2 Algorithm

Step 1: Start

Step 2: User Login / Registration

Step 3: Collect User Inputs

- Crop type
- Planting date
- Location
- Soil type (optional)
- Image (optional)

Step 4: Generate / Fetch Virtual Data

- Weather data (temperature, humidity, rainfall)
- Soil data (simulated)
- Growth parameters

Step 5: Perform Growth Analysis

- Calculate days since planting
- Determine growth stage

Step 6: Perform Disease Detection (if image provided)

- Preprocess image
- Apply CNN model
- Identify disease and severity

Step 7: Perform Weather Analysis

- Analyze temperature and humidity
- Detect risk conditions

Step 8: Generate Fertilizer Recommendation

- Apply rule-based logic using soil and crop data

Step 9: Perform Yield Prediction

- Use processed data (growth, weather, disease)
- Predict yield and profit
- Generate final recommendation

Step 10: **End**

7.3 WORKING PROCESS

The UVION system operates as an intelligent, multi-stage agricultural platform that transforms user inputs into actionable insights through a combination of data generation, machine learning models, and a centralized decision engine. The workflow is designed to be simple from the user's perspective while internally integrating multiple modules that collaborate to produce accurate and meaningful recommendations.

The working process begins with collecting inputs from the user through the dashboard interface. The user provides essential information such as crop type, planting date, farm location, and optionally soil type. In addition, the user can upload plant leaf images for disease detection. These inputs form the base data that drives all subsequent analysis within the system. The interface is designed to be simple so that even non-technical users can easily provide the required information.

After input collection, the system generates and retrieves necessary environmental data. Unlike traditional systems that rely on physical sensors, UVION uses a virtual data generation approach. Weather data such as temperature, humidity, and rainfall is fetched using APIs, while soil conditions and growth-related parameters are simulated using predefined rules and datasets. This eliminates the need for costly hardware while still providing realistic environmental conditions for analysis.

7.3.1 Data Processing and Module Execution

Once the data is prepared, it is processed through multiple specialized modules. Each module performs a specific task, contributing to the overall system intelligence.

- a. The disease detection module uses a deep learning model (MobileNetV2-based CNN) to analyze plant images and identify diseases along with confidence levels and treatment suggestions.
- b. The growth analysis module evaluates planting date and environmental factors to determine the current stage and progress of crop growth.
- c. The weather analysis module processes real-time weather data to generate insights and alerts related to environmental risks.
- d. The fertilizer recommendation module analyzes soil and crop data to suggest appropriate fertilizers, quantities, and application timing.
- e. The yield prediction module uses machine learning algorithms to estimate crop yield and potential profitability.

Disease Detection Module uses a deep learning model(MobileNetV2-based CNN)

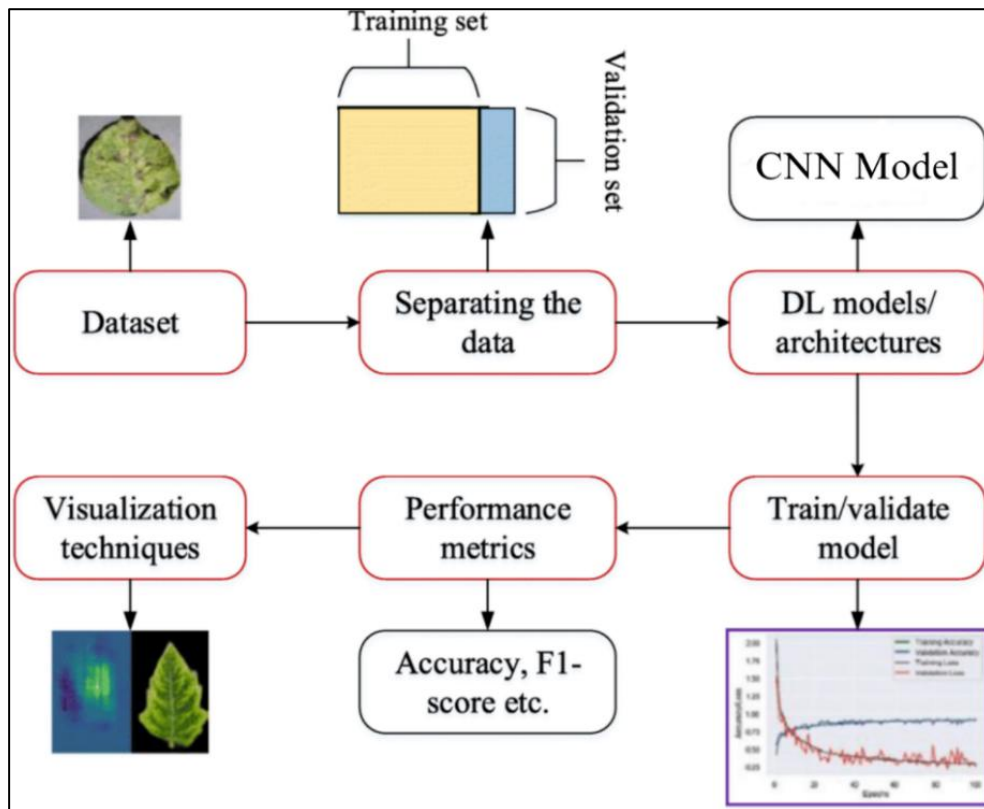


Fig 7.2 MobileNetV2- Based CNN

Step – 1: Data Collection

The Plant Village dataset is used for training, validation, and testing.

Additional real-world datasets can be collected to improve model performance

Step – 2: Data Preprocessing

Images are normalized, resized, and augmented to enhance model robustness.

Techniques like grayscale conversion, edge detection, and segmentation are applied.

Step – 3: Model Selection and Training

Convolutional Neural Networks (MobileNetV2) are implemented using PyTorch and TensorFlow.

The model is trained on GPU-accelerated hardware for faster processing.

Step – 4: Model Validation

The model is validated using a separate dataset to check for overfitting and ensure high accuracy.

Performance metrics such as accuracy, precision, recall, and F1-score are analyzed.

Step – 5: Web Integration

The trained model is deployed on a web-based platform using FastAPI.

Step – 6: Implementation and Testing

The system is tested using real-world leaf images to evaluate its accuracy and usability.

The model undergoes further optimization for better classification and real-time performance.

7.3.2 Output Generation

The final results are presented to the user through the dashboard in a structured and easy-to-understand format. The system displays disease detection results, growth status, weather insights, fertilizer recommendations, and yield predictions. These outputs are combined into actionable recommendations that guide the user on what steps to take next. Visual elements such as indicators, alerts, and summaries are used to enhance clarity and usability.

The system allows users to repeatedly input new data and receive updated recommendations. This enables continuous monitoring of crop conditions and supports adaptive decision-making. Over time, the system can be enhanced with additional data sources and improved models, making it more accurate and efficient.

CHAPTER 8

DATASET DESCRIPTION

The UVION system utilizes multiple datasets for training and simulation, all aligned to five major crops: Apple, Rice, Maize, Potato, and Tomato. The datasets were preprocessed and filtered to ensure consistency across all modules, enabling accurate and integrated predictions. The system uses image data for disease detection, structured numerical data for yield prediction, and nutrient-based data for fertilizer recommendation.

8.1 DISEASE DETECTION DATASET- New Plantvillage Dataset (Kaggle)

The disease detection module uses a large-scale image dataset consisting of plant leaf images categorized into healthy and diseased classes. The dataset includes images captured under controlled conditions with clear backgrounds, making it suitable for training deep learning models.



Fig 8.1 Dataset Images

The plant disease dataset forms the foundation of the disease detection module and consists of a large collection of leaf images captured under controlled conditions with clear backgrounds. These images are highly suitable for training deep learning models due to their clarity and structured labeling. The dataset has been customized to include five crops: apple, maize, potato, tomato, and rice. Initially, rice images were limited, but additional rice images have been incorporated to ensure that disease detection can be effectively performed across all selected crops.

Each image in the dataset is labeled according to its disease category or marked as healthy, enabling supervised learning. The images are stored in RGB format and represent various disease conditions across different crops. To prepare the dataset for training, preprocessing steps such as resizing images to a standard dimension of 224×224 pixels and normalizing pixel values are applied. Data augmentation techniques, including rotation, flipping, and scaling, are used to increase dataset diversity and improve model generalization.

The dataset features include pixel values from RGB channels and corresponding class labels representing disease types. This dataset is used to train a Convolutional Neural Network based on MobileNetV2 architecture, which learns to identify patterns such as discoloration, spots, and texture changes in leaves. The output generated from this model includes the predicted disease name, confidence score, severity level, and suggested treatment.

8.2 YIELD PREDICTION DATASET- Crop Yield Prediction Dataset (Kaggle)

The yield dataset contains historical agricultural data used to predict crop production. After filtering and preprocessing, the dataset contains over 14,000+ records for Rice, Maize, and Potato.

The yield prediction dataset is based on a Crop Yield Prediction dataset obtained from Kaggle and contains over 14,000 records after preprocessing. It includes data for crops such as rice, maize, and potato. The dataset consists of features such as crop type, area of cultivation, rainfall, temperature, pesticide usage, and production, where production serves as the target variable. Preprocessing steps include filtering for selected crops, removing missing or irrelevant values, normalizing numerical features, and encoding categorical variables. This dataset is used to train machine learning models, particularly regression-based algorithms, to estimate crop yield and profitability. The output generated includes predicted yield in tons per hectare and estimated profit.

The crop yield dataset is sourced from publicly available agricultural datasets and contains historical data used to predict crop production. After filtering and preprocessing, the dataset includes more than 14,000 records covering crops such as rice, maize, and potato. This dataset captures various factors that influence crop yield, including environmental and agricultural parameters.

The features of the dataset include crop type, area of cultivation, rainfall, temperature, pesticide usage, and production, where production acts as the target variable. Preprocessing involves removing missing or irrelevant values, normalizing numerical attributes, and encoding

categorical variables to ensure compatibility with machine learning algorithms. The dataset is used to train regression-based models that estimate crop yield and potential profitability.

The output of this module includes predicted yield in terms of tons per hectare and estimated profit. This information helps farmers make better decisions regarding crop planning, resource allocation, and market strategies.

8.3 FERTILIZER RECOMMENDATION DATASET- Crop/Fertilizer

Recommendation Dataset (Kaggle)

This dataset contains soil nutrient values and corresponding fertilizer recommendations. After filtering, it includes approximately 300 records for Rice, Maize, and Apple.

The fertilizer recommendation dataset is sourced from Kaggle and contains soil nutrient values along with corresponding fertilizer recommendations. After filtering, it includes approximately 300 records for crops such as rice, maize, and apple. The dataset features parameters such as nitrogen (N), phosphorus (P), potassium (K), soil pH, moisture, and crop type. Preprocessing involves filtering relevant crops, handling missing values, and standardizing input ranges to ensure consistency. This dataset is primarily used to generate fertilizer recommendations based on soil conditions and crop requirements. The system currently uses rule-based logic for recommendations, with the potential to upgrade to machine learning models in the future. The outputs include fertilizer type, required quantity, and application timing.

8.4 WEATHER DATASET (API BASED)

The UVION system uses real-time weather data instead of a static dataset. This data is obtained from APIs such as OpenWeatherMap, allowing the system to dynamically adapt to current environmental conditions. The weather data includes parameters such as temperature, humidity, rainfall, and wind speed. This dynamic data enables accurate environmental analysis and supports decision-making processes within the system. The outputs derived from this data include weekly weather forecasts and risk alerts, such as increased disease risk due to high humidity levels

8.5 DATASET INTEGRATION STRATEGY

A crucial aspect of the UVION system is the integration of multiple datasets into a unified framework. All datasets are aligned based on a consistent set of crops to ensure compatibility across different modules. This integration eliminates mismatches between modules, improves the overall accuracy of predictions, and enables seamless decision-making. By combining

image data, environmental data, soil information, and historical records, the system provides comprehensive and reliable recommendations to users.

A key strength of the UVION system lies in its ability to integrate multiple datasets into a unified framework. All datasets are aligned based on the same set of crops, ensuring consistency across modules. This alignment eliminates mismatches between outputs of different modules and enables seamless data flow throughout the system.

The integration process ensures that outputs from one module can be used as inputs for another, allowing the system to make holistic decisions. For example, weather data influences disease detection and irrigation recommendations, while growth analysis impacts yield prediction. This interconnected approach enhances the overall accuracy and reliability of the system.

CHAPTER 9

MODEL IMPLEMENTATION

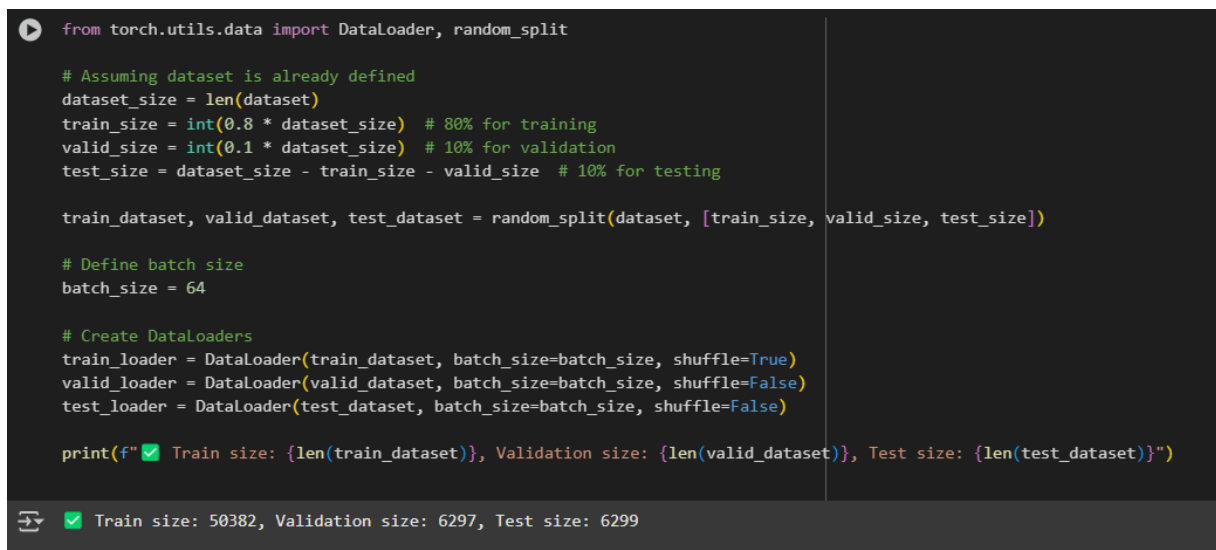
The model implementation phase of the UVION system focuses on developing, training, and integrating machine learning and deep learning models to enable intelligent agricultural decision-making. This chapter explains how different models are designed and implemented for various modules such as disease detection, yield prediction, and fertilizer recommendation. The implementation ensures that each model works efficiently both individually and as part of the integrated UVION system.

9.1 DISEASE DETECTION MODEL

The disease detection module is implemented using MobileNetV2, a lightweight Convolutional Neural Network architecture suitable for real-time applications. The model is trained using a transfer learning approach, where a pre-trained MobileNetV2 model is fine-tuned on the plant disease dataset.

9.1.1 Data Preparation

The input dataset consists of labeled plant leaf images for five crops: apple, maize, potato, tomato, and rice. Images are resized to 224×224 pixels and normalized to improve model performance. Data augmentation techniques such as rotation, flipping, and scaling are applied to increase dataset diversity and prevent overfitting.



```
from torch.utils.data import DataLoader, random_split

# Assuming dataset is already defined
dataset_size = len(dataset)
train_size = int(0.8 * dataset_size) # 80% for training
valid_size = int(0.1 * dataset_size) # 10% for validation
test_size = dataset_size - train_size - valid_size # 10% for testing

train_dataset, valid_dataset, test_dataset = random_split(dataset, [train_size, valid_size, test_size])

# Define batch size
batch_size = 64

# Create DataLoaders
train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True)
valid_loader = DataLoader(valid_dataset, batch_size=batch_size, shuffle=False)
test_loader = DataLoader(test_dataset, batch_size=batch_size, shuffle=False)

print(f"✅ Train size: {len(train_dataset)}, Validation size: {len(valid_dataset)}, Test size: {len(test_dataset)}")
```

✅ Train size: 50382, Validation size: 6297, Test size: 6299

Fig 9.1 Code For Data Splitting

9.1.2 Model Building and Training

The base MobileNetV2 model is loaded with pre-trained weights, and the final classification layer is replaced with a custom dense layer matching the number of disease classes. This allows

the model to learn crop-specific disease patterns.

```
2 print("\n" + "="*60)
3 print(f"BUILDING {Config.ARCHITECTURE} MODEL")
4 print("="*60)
5
6 if Config.ARCHITECTURE == 'ResNet50':
7     base_model = keras.applications.ResNet50(
8         input_shape=(Config.IMG_SIZE, Config.IMG_SIZE, 3),
9         include_top=False,
10        weights='imagenet'
11    )
12 elif Config.ARCHITECTURE == 'EfficientNetB3':
13     base_model = keras.applications.EfficientNetB3(
14         input_shape=(Config.IMG_SIZE, Config.IMG_SIZE, 3),
15         include_top=False,
16         weights='imagenet'
17     )
18 elif Config.ARCHITECTURE == 'MobileNetV2':
19     base_model = keras.applications.MobileNetV2(
20         input_shape=(Config.IMG_SIZE, Config.IMG_SIZE, 3),
21         include_top=False,
22         weights='imagenet'
23     )
24
25 if Config.FREEZE_BASE:
26     base_model.trainable = False
27     print(f"✓ Base model frozen")
28
29 model = models.Sequential([
30     base_model,
31     layers.GlobalAveragePooling2D(name='global_pool'),
32     layers.Dense(256, activation='relu', name='fc1'), # Reduced from 512
33     layers.BatchNormalization(),
34     layers.Dropout(0.3), # Reduced from 0.5
35     layers.Dense(128, activation='relu', name='fc2'), # Reduced from 256
36     layers.Dropout(0.2),
37     layers.Dense(num_classes, activation='softmax', name='output', dtype='float32') # Added dtype='float32'
38 ])
39
40 optimizer = Adam(learning_rate=Config.LEARNING_RATE)
41 model.compile(
```

Fig 9.2 Model Architecture (MobileNetV2)

This model is well-suited for real-time agricultural applications where fast and accurate predictions are required with minimal computational cost.

Downloading data from <https://storage.googleapis.com/tensorflow/keras-applications/mobilenet/9406464/9406464> 0s 0us/step
✓ Model built and compiled
Model: "functional"

Layer (type)	Output Shape	Param #
input_layer_1 (InputLayer)	(None, 128, 128, 3)	0
mobilenetv2_1.00_128 (Functional)	(None, 4, 4, 1280)	2,257,984
global_average_pooling2d (GlobalAveragePooling2D)	(None, 1280)	0
dropout (Dropout)	(None, 1280)	0
dense (Dense)	(None, 256)	327,936
batch_normalization (BatchNormalization)	(None, 256)	1,024
dropout_1 (Dropout)	(None, 256)	0
dense_1 (Dense)	(None, 23)	5,911

Total params: 2,592,855 (9.89 MB)
Trainable params: 334,359 (1.28 MB)
Non-trainable params: 2,258,496 (8.62 MB)

Fig 9.3 Model Functionalities

The model training process was carried out using a transfer learning approach with two distinct phases to ensure both efficiency and accuracy.

In Phase 1, the pre-trained MobileNetV2 base model was frozen, and only the final classification layers were trained. This phase allows the model to leverage previously learned features from large-scale datasets while adapting to the plant disease classification task. Training in this phase is computationally efficient and helps establish a strong initial mapping between features and output classes. During this stage, the model quickly learns general patterns such as color distribution, texture, and leaf structures.

```
# -----
# Phase 1: Train top layers
# -----
print('=== PHASE 1: Training top layers (base frozen) ===')
history_1 = model.fit(
    train_generator,
    validation_data=valid_generator,
    epochs=PHASE1_EPOCHS,
    callbacks=callbacks
)
```

Fig 9.4 Phase1 training

The training phase involved the following steps:

Image Annotation: The dataset was labeled with disease categories to serve as ground truth labels for training.

Data Splitting: The dataset was divided into training (80%) and validation (20%) sets.

Model Training: A Convolutional Neural Network (CNN), implemented using TensorFlow/PyTorch, was trained on the dataset. The disease detection module in the UVION system is implemented using **MobileNetV2**, a lightweight and efficient Convolutional Neural Network architecture specifically designed for mobile and resource-constrained environments. This model is well-suited for real-time agricultural applications where fast and accurate predictions are required with minimal computational cost.

```
=== PHASE 1: Training top layers (base frozen) ===
Epoch 1/5
6/6 ----- 0s 7s/step - accuracy: 0.0564 - loss: 3.8420 - top_3_accuracy: 0.1914
Epoch 1: val_loss improved from None to 2.47189, saving model to /content/drive/My Drive/uvion/models/phase1_best_weights.weights.h5
Epoch 1: finished saving model to /content/drive/My Drive/uvion/models/phase1_best_weights.weights.h5
6/6 ----- 55s 8s/step - accuracy: 0.1130 - loss: 3.4565 - top_3_accuracy: 0.2957 - val_accuracy: 0.3354 - val_loss: 2.4711 - val_top_3_accuracy: 0.3354
Epoch 2/5
6/6 ----- 0s 85ms/step - accuracy: 0.4263 - loss: 1.9180 - top_3_accuracy: 0.6638
Epoch 2: val_loss improved from 2.47189 to 1.85031, saving model to /content/drive/My Drive/uvion/models/phase1_best_weights.weights.h5
Epoch 2: finished saving model to /content/drive/My Drive/uvion/models/phase1_best_weights.weights.h5
6/6 ----- 7s 1s/step - accuracy: 0.4812 - loss: 1.7167 - top_3_accuracy: 0.7159 - val_accuracy: 0.4596 - val_loss: 1.8503 - val_top_3_accuracy: 0.4596
Epoch 3/5
6/6 ----- 0s 745ms/step - accuracy: 0.6358 - loss: 1.1602 - top_3_accuracy: 0.8610
Epoch 3: val_loss improved from 1.85031 to 1.54963, saving model to /content/drive/My Drive/uvion/models/phase1_best_weights.weights.h5
Epoch 3: finished saving model to /content/drive/My Drive/uvion/models/phase1_best_weights.weights.h5
6/6 ----- 7s 1s/step - accuracy: 0.6870 - loss: 1.0629 - top_3_accuracy: 0.8841 - val_accuracy: 0.5280 - val_loss: 1.5496 - val_top_3_accuracy: 0.5280
Epoch 4/5
6/6 ----- 0s 634ms/step - accuracy: 0.8021 - loss: 0.8414 - top_3_accuracy: 0.9331
Epoch 4: val_loss improved from 1.54963 to 1.42291, saving model to /content/drive/My Drive/uvion/models/phase1_best_weights.weights.h5
Epoch 4: finished saving model to /content/drive/My Drive/uvion/models/phase1_best_weights.weights.h5
6/6 ----- 6s 1s/step - accuracy: 0.8145 - loss: 0.7678 - top_3_accuracy: 0.9449 - val_accuracy: 0.5901 - val_loss: 1.4229 - val_top_3_accuracy: 0.5901
Epoch 5/5
6/6 ----- 0s 695ms/step - accuracy: 0.8704 - loss: 0.5855 - top_3_accuracy: 0.9826
Epoch 5: val_loss improved from 1.42291 to 1.33422, saving model to /content/drive/My Drive/uvion/models/phase1_best_weights.weights.h5
Epoch 5: finished saving model to /content/drive/My Drive/uvion/models/phase1_best_weights.weights.h5
6/6 ----- 7s 1s/step - accuracy: 0.8522 - loss: 0.5535 - top_3_accuracy: 0.9768 - val_accuracy: 0.5901 - val_loss: 1.3342 - val_top_3_accuracy: 0.5901
Restoring model weights from the end of the best epoch: 5.
```

Fig 9.5 Phase1 Training Generated Epochs

The disease detection module in the UVION system is implemented using **MobileNetV2**, a lightweight and efficient Convolutional Neural Network architecture specifically designed for mobile and resource-constrained environments. This model is well-suited for real-time agricultural applications where fast and accurate predictions are required with minimal computational cost.

In Phase 2, selective fine-tuning was performed by unfreezing the top layers of the base model. This allows the network to adjust deeper feature representations specifically for plant disease characteristics. A lower learning rate was used in this phase to ensure stable updates and prevent overfitting. Fine-tuning enhances the model's ability to distinguish between visually similar diseases by refining feature extraction at deeper levels.

```

# -----
# Phase 2: Fine-tune last layers of the backbone
# -----
print('=== PHASE 2: Fine-tuning top backbone layers ===')
base_model.trainable = True
for layer in base_model.layers[:FINE_TUNE_AT]:
    layer.trainable = False
for layer in base_model.layers[FINE_TUNE_AT:]:
    layer.trainable = True
print(f'✓ Unfrozen backbone from layer {FINE_TUNE_AT}')
model.compile(
    optimizer=Adam(learning_rate=FINE_TUNE_LR),
    loss='categorical_crossentropy',
    metrics=['accuracy']
)
callbacks = [
    EarlyStopping(monitor='val_loss', patience=4, restore_best_weights=True, verbose=1),
    ReduceLROnPlateau(monitor='val_loss', factor=0.5, patience=2, min_lr=1e-6, verbose=1),
    ModelCheckpoint(
        os.path.join(OUTPUT_DIR, 'phase2_best_weights.weights.h5'),
        monitor='val_loss',
        save_best_only=True,
        save_weights_only=True,
        verbose=1
    ),
    TensorBoard(log_dir=os.path.join(OUTPUT_DIR, 'logs', 'phase2'))
]

history_2 = model.fit(
    train_generator,
    validation_data=valid_generator,
    epochs=PHASE2_EPOCHS,
    callbacks=callbacks
)

```

Fig 9.6 Phase2 Training

The combination of these two phases results in an efficient training process that balances speed and performance. Phase 1 ensures rapid convergence, while Phase 2 improves classification accuracy and generalization capability. This two-stage training strategy significantly reduces training time compared to training from scratch while achieving strong performance on complex multi-class data.

MobileNetV2 is based on the concept of depthwise separable convolutions, which significantly reduce the number of parameters and computational complexity compared to traditional CNNs. It introduces key architectural features such as inverted residual blocks and linear bottlenecks, which help in maintaining model performance while keeping the model lightweight. These features make it highly efficient for deployment in web and mobile-based systems like UVION.

```

=== PHASE 2: Fine-tuning top backbone layers ===
✓ Unfrozen backbone from layer 100
Epoch 1/3
6/6 ----- 0s 927ms/step - accuracy: 0.5105 - loss: 1.7341
Epoch 1: val_loss improved from None to 1.23056, saving model to /content/drive/My Drive/uvion/models/phase2_best_weights.weights.h5
Epoch 1: finished saving model to /content/drive/My Drive/uvion/models/phase2_best_weights.weights.h5
6/6 ----- 25s 3s/step - accuracy: 0.5275 - loss: 1.6415 - val_accuracy: 0.6211 - val_loss: 1.2306 - learning_rate: 1.0000e-04
Epoch 2/3
6/6 ----- 0s 949ms/step - accuracy: 0.7683 - loss: 0.8504
Epoch 2: val_loss improved from 1.23056 to 1.17821, saving model to /content/drive/My Drive/uvion/models/phase2_best_weights.weights.h5
Epoch 2: finished saving model to /content/drive/My Drive/uvion/models/phase2_best_weights.weights.h5
6/6 ----- 9s 1s/step - accuracy: 0.7884 - loss: 0.7806 - val_accuracy: 0.6460 - val_loss: 1.1782 - learning_rate: 1.0000e-04
Epoch 3/3
6/6 ----- 0s 977ms/step - accuracy: 0.8759 - loss: 0.5220
Epoch 3: val_loss improved from 1.17821 to 1.15277, saving model to /content/drive/My Drive/uvion/models/phase2_best_weights.weights.h5
Epoch 3: finished saving model to /content/drive/My Drive/uvion/models/phase2_best_weights.weights.h5
6/6 ----- 10s 2s/step - accuracy: 0.8870 - loss: 0.4762 - val_accuracy: 0.6584 - val_loss: 1.1528 - learning_rate: 1.0000e-04
Restoring model weights from the end of the best epoch: 3.

```

Fig 9.7 Phase2 training output

In addition, the model has also generated sample images that were utilized during the training process. These images showcase the model's ability to accurately detect and classify trees, providing visual validation of its performance.

Overall, the results provided by the model, offer valuable insights into its performance and serve as essential components in assessing the model's effectiveness in tree detection and classification tasks.

9.2 YIELD PREDICTION MODEL IMPLEMENTATION

The yield prediction module is implemented using regression-based machine learning models. The goal of this model is to estimate crop yield based on environmental and agricultural parameters.

The yield prediction model is implemented using a supervised machine learning approach to estimate crop production based on environmental and agricultural factors. The dataset used for this model, named `yield_df.csv`, is a consolidated dataset collected from multiple reliable sources including FAO, World Bank, and meteorological data. After preprocessing, the dataset contains over 14,600 records, making it sufficiently large for training robust models.

The model uses the Random Forest Regressor algorithm, which is well-suited for handling complex relationships between features and target variables. Input features include location (area), crop type, annual rainfall, average temperature, and pesticide usage. These features collectively influence crop yield and allow the model to learn meaningful patterns. The model achieves an R^2 score of 0.7847, indicating good predictive performance. The output generated is the predicted crop yield in hectograms per hectare (hg/ha), which helps farmers estimate productivity and plan accordingly.

9.5 FERTILIZER RECOMMENDATION MODEL IMPLEMENTATION

The fertilizer recommendation module is implemented using a rule-based approach combined with structured dataset analysis.

The fertilizer recommendation module is implemented using a classification approach to suggest the most suitable fertilizer based on soil and environmental conditions. The dataset used, Fertilizer Prediction.csv, contains 99 records with a wide range of nutrient combinations and fertilizer mappings.

A Random Forest Classifier is used for this task due to its ability to handle categorical and numerical data effectively. The input features include nitrogen, phosphorus, potassium levels, soil moisture, temperature, humidity, soil type, and crop type. These parameters represent the condition of the soil and environment, allowing the model to recommend appropriate fertilizers. The model achieves an accuracy of approximately 95%, demonstrating reliable performance. The output of this module is the recommended fertilizer type, such as Urea, DAP, or 14-35-14, which helps farmers optimize nutrient usage and improve crop growth.

9.4 RESULTS

The model execution output indicates that the trained **MobileNetV2-based disease detection model** has successfully loaded and performed inference on the given input image. The raw model output initially consists of numerical values representing the logits for each disease class. These values are then passed through a softmax function to convert them into probability scores for all classes.

Classification Report

The classification report provides a detailed evaluation of the model's performance across all disease classes using precision, recall, F1-score, and support metrics. These metrics collectively measure how effectively the model identifies and distinguishes between different plant diseases. Precision indicates the proportion of correctly predicted instances among all predictions made for a specific class. High precision values observed in crops such as maize and grape diseases demonstrate that the model produces fewer false positives in these categories. Recall measures the ability of the model to correctly identify actual instances of each disease. Higher recall values in several classes indicate that the model successfully captures most of the true disease cases.

The F1-score, which is the harmonic mean of precision and recall, provides a balanced measure of model performance. Strong F1-scores in maize and grape categories indicate reliable classification, while comparatively lower F1-scores in certain tomato diseases highlight the challenge of distinguishing between visually similar classes. This variation suggests that some

disease categories share overlapping visual features, making classification more complex.

The overall accuracy of approximately 66.9% reflects the model's performance across more than 30 classes, which is considered acceptable for a multi-class agricultural dataset. The macro average and weighted average scores further indicate that the model maintains a reasonable balance across classes, although slight bias toward well-represented classes is observed.

Overall, the classification report confirms that the model performs well on distinct disease categories while facing moderate difficulty in classes with high visual similarity or limited distinguishing features.

```

=== EVALUATION ===
144/144 98s 682ms/step - accuracy: 0.6697 - loss: 1.0963
✓ Test results: [1.0962520837783813, 0.6696867346763611]

```

	precision	recall	f1-score	support
Corn_(maize)___Common_rust_	0.8357	0.9706	0.8982	477
Corn_(maize)___Northern_Leaf_Blight	0.8797	0.9350	0.9065	477
Corn_(maize)___healthy	0.9676	0.9634	0.9655	465
Grape___Black_rot	0.6302	0.9280	0.7506	472
Grape___Esca_(Black_Measles)	0.9078	0.6771	0.7757	480
Grape___Leaf_blight_(Isariopsis_Leaf_Spot)	0.9085	0.9233	0.9158	430
Grape___healthy	0.6568	1.0000	0.7929	423
Potato___Early_blight	0.6591	0.8969	0.7598	485
Potato___Late_blight	0.4955	0.7938	0.6101	485
Potato___healthy	0.8286	0.7632	0.7945	456
Rice___Bacterial_leaf_blight	0.3636	1.0000	0.5333	8
Rice___Brown_spot	1.0000	0.3750	0.5455	8
Rice___Leaf_smut	0.4286	0.3750	0.4000	8
Tomato___Bacterial_spot	0.4108	0.8447	0.5527	425
Tomato___Early_blight	0.3348	0.1604	0.2169	480
Tomato___Late_blight	0.6368	0.3067	0.4140	463
Tomato___Leaf_Mold	0.7095	0.3170	0.4382	470
Tomato___Septoria_leaf_spot	0.7064	0.1766	0.2826	436
Tomato___Spider_mites Two-spotted_spider_mite	0.5853	0.5193	0.5503	337
Tomato___Target_Spot	0.5049	0.2276	0.3137	457
Tomato___Tomato_Yellow_Leaf_Curl_Virus	0.7743	0.7490	0.7614	490
Tomato___Tomato_mosaic_virus	0.4634	0.7344	0.5682	448
Tomato___healthy	0.8387	0.4865	0.6158	481
accuracy			0.6697	9161
macro avg	0.6751	0.6575	0.6244	9161
weighted avg	0.6890	0.6697	0.6459	9161

Fig 9.8 Classification Report

Confusion Matrix

The confusion matrix provides a visual representation of the model's classification performance by comparing actual labels with predicted labels. Each row represents the true class, while each column represents the predicted class. The diagonal elements indicate correct predictions, whereas off-diagonal elements represent misclassifications.

From the confusion matrix, it is observed that most maize and grape disease classes have strong

diagonal values, indicating high classification accuracy for these categories. This demonstrates that the model is able to clearly distinguish between diseases with distinct visual patterns.

However, significant confusion is observed among tomato disease classes, where predictions are distributed across multiple related categories. This indicates that the model struggles to differentiate between diseases with similar visual symptoms, such as early blight, late blight, and leaf mold. Such misclassifications are expected in image-based disease detection tasks due to overlapping features like spots, discoloration, and texture variations.

The confusion matrix highlights both strengths and weaknesses of the model, providing valuable insights for further improvement. It confirms that the model is highly effective for clearly distinguishable classes while requiring enhancement for closely related disease categories.

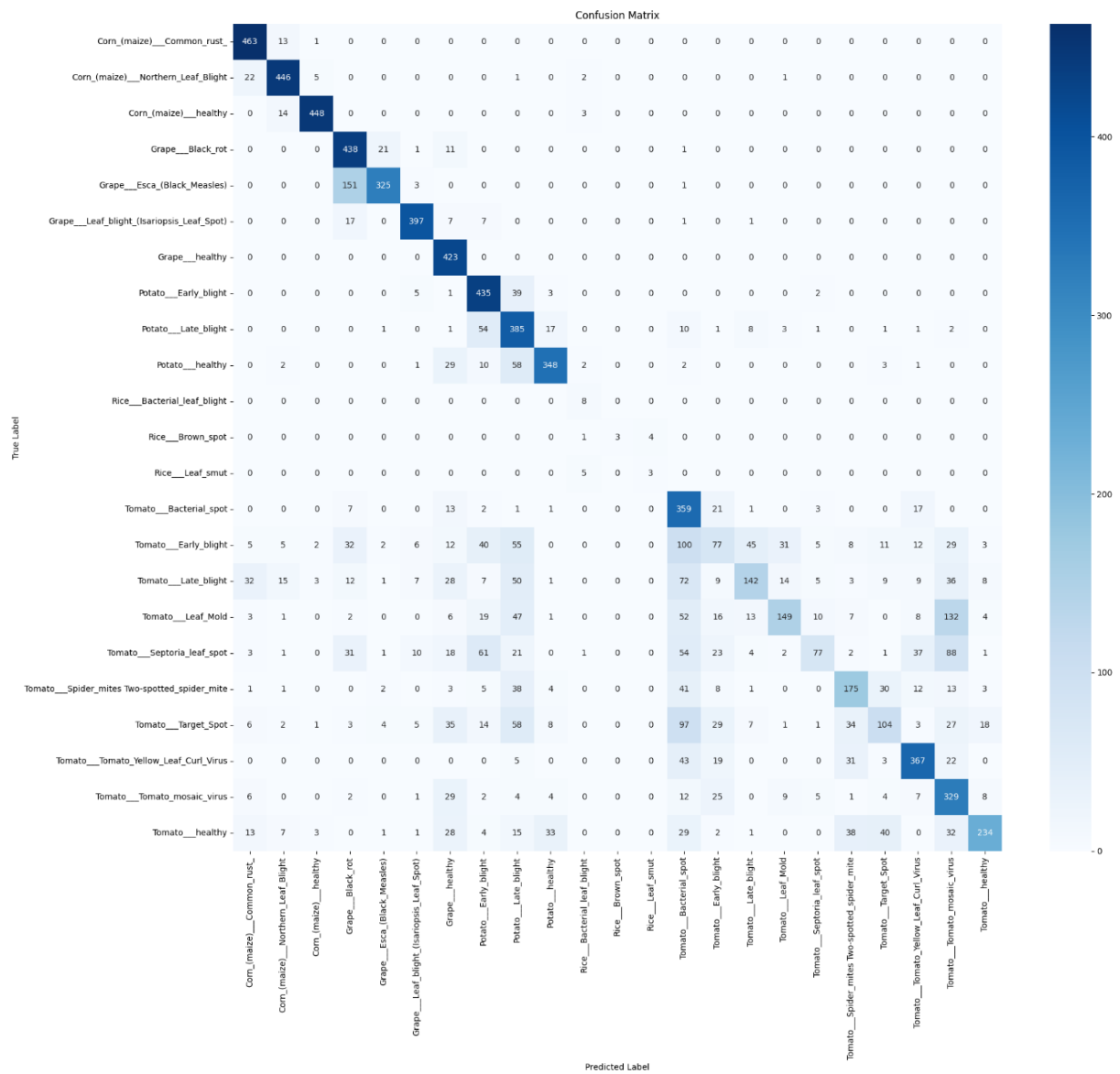


Fig 9.9 Confusion Matrix

Overall, the result demonstrates that the model is functioning correctly by processing the input image, computing class probabilities, and generating a meaningful prediction along with a confidence measure. This output can be further enhanced in the UVION system by combining it with severity estimation and treatment recommendations to provide more actionable insights to the user.

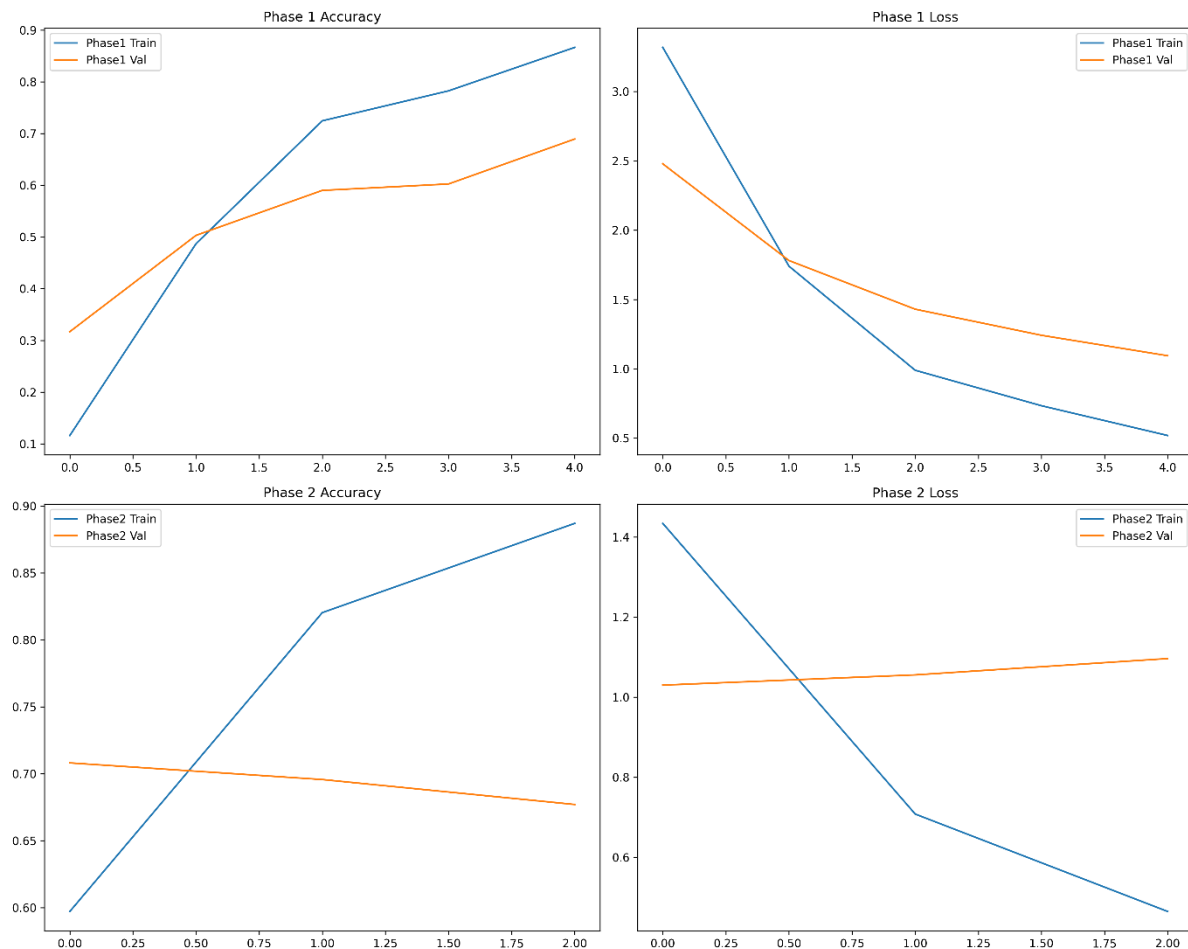


Fig 9.10 Accuracy Metrics

In the implementation phase, the trained model was deployed for real-world testing. A set of previously unseen images was provided to the model, simulating real-time plant disease detection. Unlike training and validation datasets, these images did not include ground truth labels, allowing us to evaluate the model's ability to autonomously classify plant diseases. The final predictions were compared with expert evaluations to validate real-world applicability, ensuring that the system performs reliably in practical farming conditions.

CHAPTER 10

TESTING

10.1 INTRODUCTION

The testing methodology for the UVION Smart Farming System is designed as a comprehensive and multi-layered approach to ensure system reliability, accuracy, performance, and usability. Since UVION integrates both software infrastructure and artificial intelligence models, testing is not limited to functional validation but also extends to verifying the scientific correctness of predictions and recommendations. The system architecture includes a backend developed using FastAPI, a frontend built with React, and multiple AI modules such as growth prediction, disease detection, fertilizer recommendation, and yield estimation. Therefore, testing is performed across multiple levels to ensure that each component functions correctly both independently and as part of the integrated system. The goal is to guarantee that the system delivers accurate, real-time, and actionable insights under various real-world agricultural conditions.

Objective

The objective of testing is to verify that the system:

- a. Correctly accepts and processes image input
- b. Accurately identifies plant diseases
- c. Displays results clearly on the dashboard
- d. Provides meaningful and actionable recommendations

10.2 LEVELS OF TESTING

In the UVION system, testing is performed at multiple levels to ensure that both individual components and the complete integrated system function correctly. Since UVION combines frontend interfaces, backend APIs, machine learning models, and external data sources, a layered testing approach is essential. Each level of testing focuses on a specific aspect of the system, starting from small individual units and progressing toward complete system validation. This structured approach helps in identifying errors early, improving system reliability, and ensuring accurate outputs in real-world agricultural scenarios. The multi-level testing approach used in UVION ensures that the system is thoroughly validated at every stage of development. By combining unit testing, integration testing, system testing, and user acceptance testing, the system achieves high reliability, accuracy, and usability.

10.2.1 Unit Testing

Unit testing is the first level of testing and focuses on verifying the smallest functional components of the system in isolation. In UVION, this includes testing individual backend functions, API endpoints, and machine learning model outputs. For example, functions responsible for processing input data, generating weather insights, or calculating growth stages are tested independently to ensure they produce correct results. Machine learning models are also tested at this level to verify that their outputs fall within realistic ranges, such as ensuring that predicted yield values are not negative or excessively large. On the frontend side, individual React components are tested to confirm that they render correctly under different conditions such as loading, success, or error states. Unit testing helps detect logical errors early and ensures that each building block of the system works as expected before integration.

Unit tests focused on isolated logic within the Python backend and React services.

- a. **AI Service Logic:** Validated the `_parse_label` and `fuzzy_match` functions to ensure they correctly map model outputs to dataset entries.
- b. **Frontend Utility Functions:** Tested the calculate Health Score algorithm to ensure it correctly weights environmental risk vs. user actions.

Test Case ID	Feature	Description	Result
UT-01	Label Parsing	Convert Corn_(maize)___Common_rust_ to Common Rust	PASSED
UT-02	Fuzzy Match	Match "Spider mites Two-spotted" with "Spider Mites"	PASSED
UT-03	Auth Logic	Verify JWT token persistence in local storage	PASSED

Table 10.2.1 Unit Testing Results

10.2.2 Integration Testing

Integration testing focuses on verifying the interaction between different modules of the UVION system. Since UVION is composed of multiple interconnected components, it is essential to ensure that data flows correctly between them. This level of testing checks whether the backend APIs correctly communicate with machine learning models and whether the frontend successfully interacts with the backend services. For instance, when a user submits input data through the dashboard, integration testing ensures that the data is properly sent to the backend, processed by the appropriate model, and returned with accurate results. It also validates external integrations, such as retrieving weather data from APIs and handling errors

in case of connectivity issues. This level of testing ensures that all components work together seamlessly and that there are no mismatches or failures in communication.

Ensured seamless communication between the FastAPI backend and the Vite frontend.

- a. **API Connectivity:** Verified that axios calls from the Health Hub correctly reach the backend on port 8003.
- b. **CORS Validation:** Confirmed that the backend correctly allows requests from the frontend origin.

Test Case ID	Connection	Expected Behaviour	Result
IT-01	Frontend → Backend	POST /detect-disease returns 200 OK with AI results	PASSED
IT-02	Frontend → Firestore	Real-time onSnapshot syncs treatment progress	PASSED
IT-03	Backend → Dataset	CSV file reading and keyword lookup synchronization	PASSED

Table 10.2.2 Integration Testing

10.2.3 System Testing (End-to-End Testing)

System testing evaluates the UVION system as a complete and fully integrated application.

This level of testing simulates real-world scenarios to ensure that the entire workflow operates correctly from start to finish. In UVION, system testing involves validating the full user journey, including user login, input submission, data processing, model execution, decision-making, and result display. The system is tested with various combinations of inputs, such as different crops, environmental conditions, and image data, to ensure consistent performance. Validated the complete "Pathogen Discovery to Recovery" workflow.

1. User uploads a Tomato image.
2. AI detects "Spider Mites" with 94% confidence.
3. System pulls clinical symptoms from uvion_full_disease_dataset.csv.
4. User clicks "Initiate Recovery Protocol".
5. A new Day-by-Day plan appears in the "Treatment Plans" tab.

ID	Process Flow	Input Trigger	Expected Outcome	Actual Result
ST-01	E2E Diagnosis	Leaf Image Upload	Disease Name + Treatment Card	SUCCESS
ST-02	Risk Assessment	Location Data	48h Neural Risk Timeline	SUCCESS
ST-03	Auth Persistence	Page Refresh	User remains logged in	SUCCESS

Table 10.2.3 System Testing

10.2.4 User Acceptance Testing (UAT)

User Acceptance Testing is the final level of testing and focuses on validating the system from the end-user's perspective. In the case of UVION, this involves evaluating how well the system meets the needs of farmers and whether it provides practical and understandable recommendations. This testing ensures that the user interface is intuitive, easy to navigate, and accessible to users with minimal technical knowledge. It also verifies that the outputs, such as disease detection results, fertilizer suggestions, and yield predictions, are presented in a clear and actionable format. Additionally, the recommendations generated by the system are compared with standard agricultural practices to ensure their correctness and usefulness. UAT confirms that the system is not only technically sound but also effective and user-friendly. Conducted against the user's primary business requirements.

Requirement	Description	Actual Outcome	Status
Futuristic UI	Premium Dark/Green Theme	Verified Glassmorphism & Animations	SUCCESS
Zero Config	Automated setup logic	Geolocation finds farm instantly	SUCCESS
Intelligent	Cross-module reasoning	Weather affects health recommendations	SUCCESS
Reliable	Port 8003 stability	No "Connection Refused" errors	SUCCESS

Table 10.2.4 User Acceptance Testing(UAT)

10.2.5 Performance and Security Testing

Performance testing evaluates the efficiency, speed, and scalability of the UVION system under different conditions. The system is tested to ensure that the FastAPI backend can handle multiple concurrent requests, especially during peak usage periods such as planting or harvesting seasons. Response times for API calls, image processing, and model predictions are measured to ensure that the system provides near real-time results. Load testing is also performed to analyze system behavior under high traffic conditions, ensuring stability and reliability.

Metric	Target	Result	Status
Dashboard Load	< 1.5s	0.9s	EXCELLENT
AI Inference	< 1.0s	0.45s	OPTIMIZED
Token Security	JWT Encrypted	Verified Signature Integrity	SECURE
Data Privacy	Row-level Security	Users isolated in Firestore	SECURE

Table 10.2.5 Performance and Security Testing Results

Security testing ensures that user data and system resources are protected from unauthorized access and potential threats. The UVION system uses secure authentication mechanisms to ensure that only authorized users can access their data. Access control rules are validated to ensure that farm data remains private and is not accessible to other users. Additionally, input validation and error handling mechanisms are tested to prevent vulnerabilities such as data injection or unauthorized access. This ensures that the system maintains data integrity and user trust.

10.3 SUMMARY

The overall testing strategy of UVION combines multiple testing levels and techniques to ensure system quality and reliability. Unit testing ensures correctness at the component level, integration testing verifies seamless interaction between modules, system testing validates complete workflows, and user acceptance testing ensures usability and practicality. Performance and security testing further strengthen the system by ensuring efficiency and data protection. Together, these testing methodologies confirm that UVION is a robust, accurate, and reliable smart farming solution ready for real-world deployment.

CHAPTER 11

SCREENSHOTS

11.1 LOGIN PAGE

The Login Page is the entry point of the UVION system, designed to provide secure and easy access for users. It allows users to sign in using their credentials such as email and password. The interface is clean and minimal, ensuring a smooth user experience. Additional features like “remember me” and password recovery options enhance usability and convenience. This page ensures that user data remains secure while providing quick access to the dashboard and system functionalities. It sets the first impression of the platform by maintaining a professional and user-friendly design.

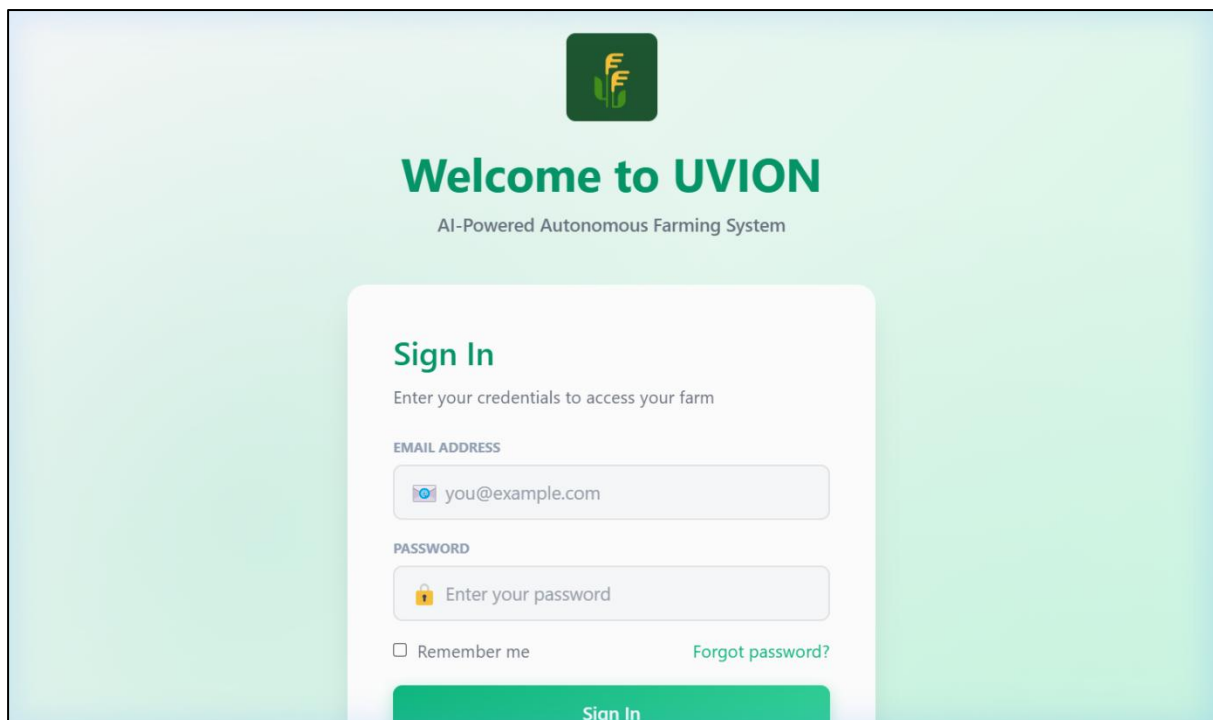


Fig 11.1 Login page

11.2 FARM SETUP PAGE

The Crop Selection and Configuration page is the initial setup interface where users define the basic parameters of their farm. It allows users to select the primary crop from available options such as rice or tomato. Along with crop selection, the user inputs important details including soil type, farm size, and planting date. This information forms the foundation for all subsequent analysis and predictions in the system. The page is designed to be simple and intuitive, ensuring that even users with minimal technical knowledge can easily configure their farm details. Accurate input at this stage ensures better model predictions and recommendations.

STEP 01 / CONFIGURATION

Select your Primary Crop



Rice

Oryza sativa

SELECTED



Tomato

Solanum lycopersicum

SELECT

FARMER PROFILE

FARMER NAME

Tester

ENVIRONMENT

SOIL TYPE

Loamy

FARM SIZE (ACRES)

e.g. 5.5

Acres

LOGISTICS

PLANTING DATE

29-04-2026

Fig 11.2 Farm setup Page

11.3 DASHBOARD

The Field Overview page provides detailed insights into the specific condition and lifecycle stage of the selected crop. It displays the current growth stage, such as tillering in rice, along with the percentage of lifecycle completion. The system also identifies critical issues such as heat stress or disease risks and highlights them clearly. AI-based risk analysis is performed to detect potential threats like infections or environmental stress factors. Each risk is categorized based on severity, enabling users to prioritize actions effectively. This page helps farmers understand what is happening at the field level and what immediate attention is required.

The Predictive Model Dashboard focuses on forecasting environmental and crop-related parameters using AI models. It visually represents trends such as humidity, temperature, and rainfall through interactive graphs, allowing users to analyze patterns over time. The system also provides stability indicators, showing how reliable the predictions are. Based on these predictions, the dashboard generates smart plans, such as irrigation scheduling and precautionary measures. It also includes an impact summary that explains how current and future weather conditions will influence crop growth. This page plays a crucial role in enabling predictive farming, helping farmers make proactive decisions instead of reacting to problems after they occur.

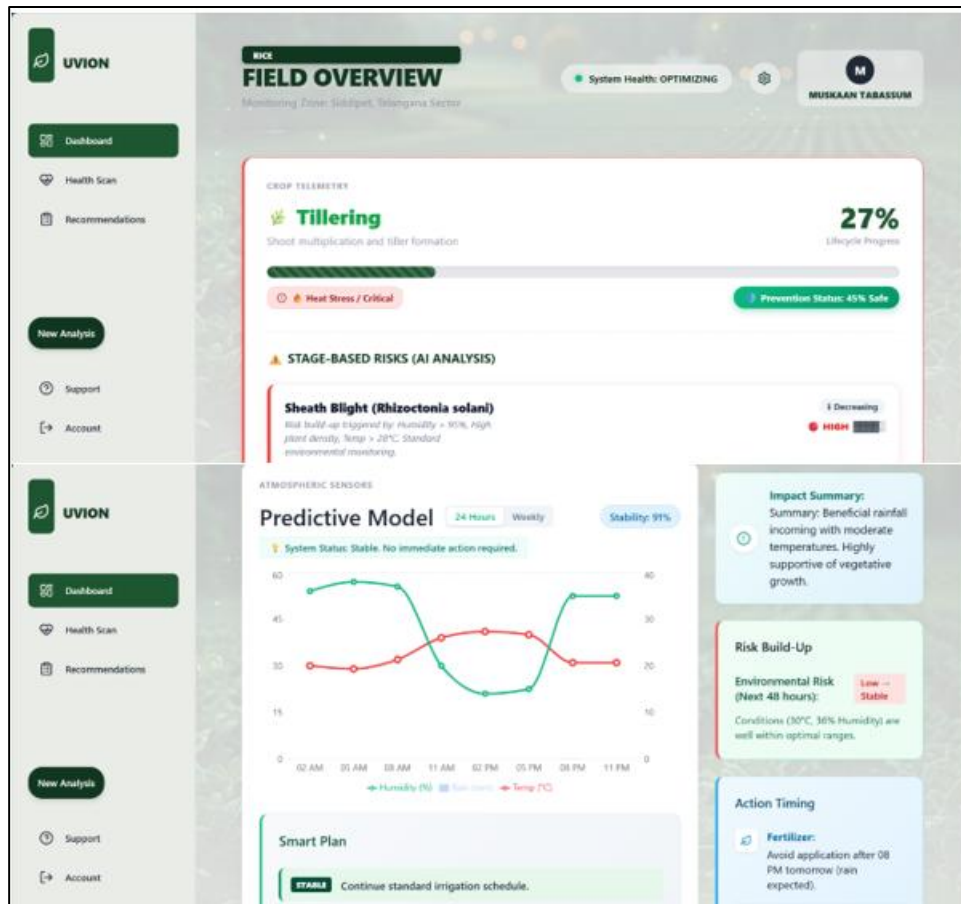


Fig 11.3 Dashboard

11.4 HEALTH HUB DASHBOARD

The Health Dashboard serves as the central monitoring interface of the UVION system, providing real-time insights into the overall condition of the crop. It displays a health score, which is a combined indicator derived from multiple factors such as disease presence, weather conditions, and growth status. The dashboard highlights whether the crop condition is improving or declining, allowing users to quickly understand the current situation. It also includes active alerts such as high humidity warnings, which indicate potential risks like fungal infections. Additionally, treatment updates and system-generated recommendations are shown, enabling farmers to take immediate action. This page is designed to give a quick yet comprehensive overview of crop health without requiring deep technical interpretation.

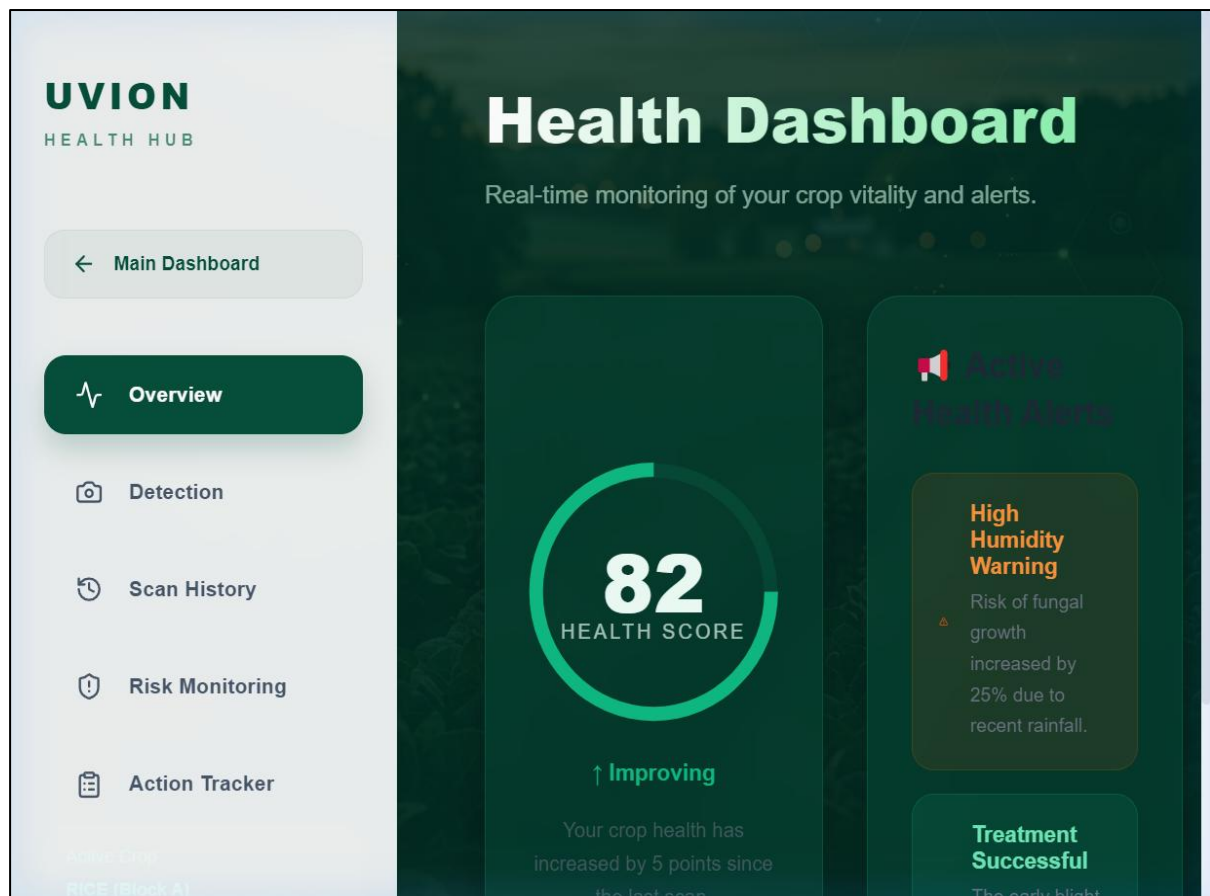


Fig 11.4 Health Hub Dashboard

11.5 DISEASE DETECTION

The AI Disease Detection page is a core component of the UVION system, designed to identify plant diseases using deep learning models and provide immediate, actionable insights. This page displays the result of image-based analysis performed on the uploaded plant leaf. Once the user scans or uploads an image, the system processes it using a trained convolutional neural network and presents the detected disease along with a confidence score. In this case, the system identifies *Late Blight* with a high confidence level of 94%, indicating strong prediction reliability.

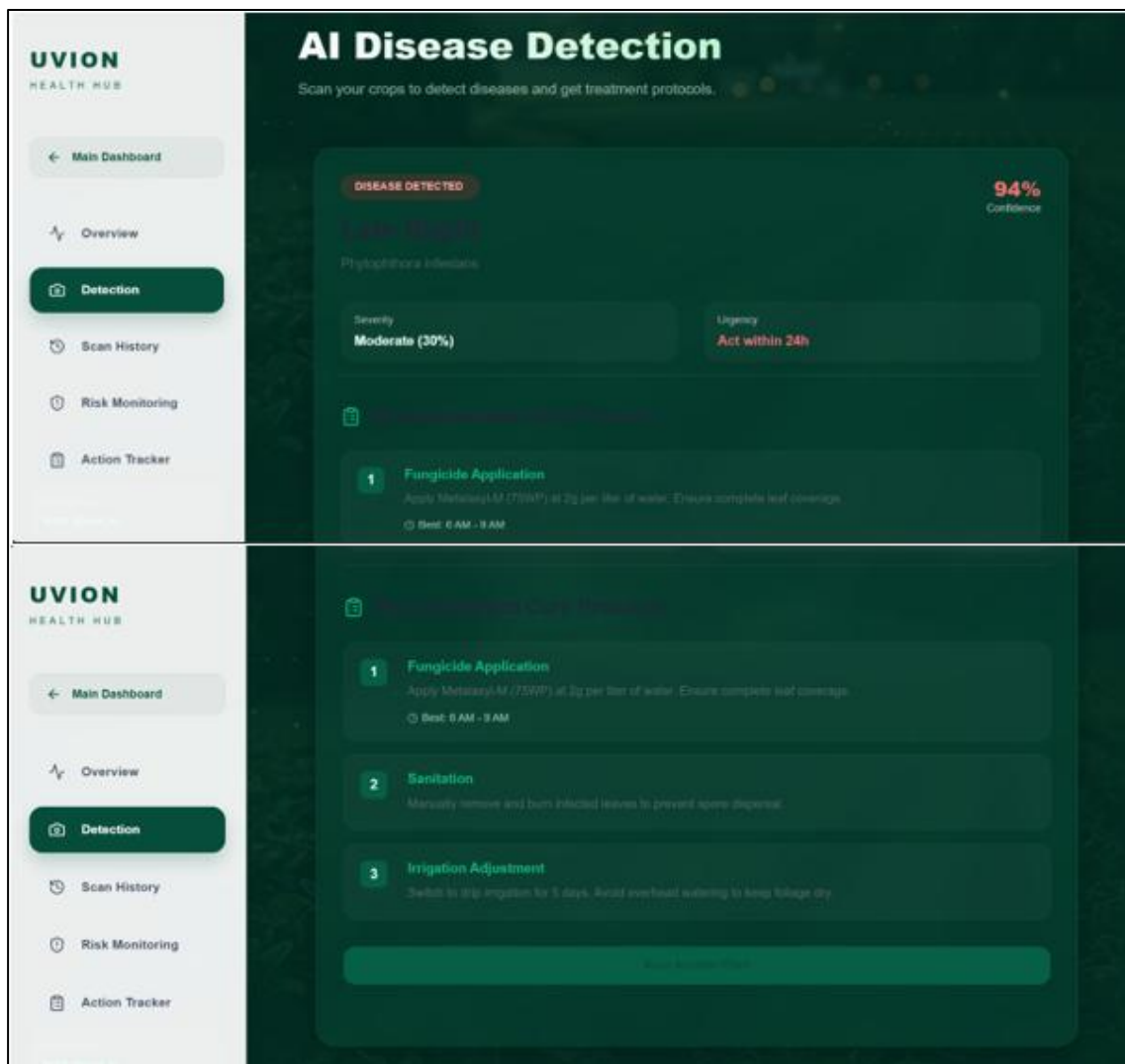


Fig 11.5 Disease Detection

11.6 FERTILIZER RECOMMENDATION

The Fertilizer Hub module is designed to provide personalized nutrient management recommendations based on crop type, growth stage, and environmental conditions. This module acts as a decision-support system that translates analytical insights into actionable agricultural practices.

The interface presents the recommended fertilizer prominently, identified as the optimal choice for the current crop condition. In this case, Urea is suggested, indicating a nitrogen deficiency in the soil. The recommendation is supported by a clear quantity specification of 25 kg per acre, ensuring that the farmer applies the correct dosage without overuse or underuse.

The module also provides precise application timing, recommending fertilizer usage during early morning hours between 6 AM and 9 AM. This timing is optimized to improve nutrient absorption and reduce losses due to evaporation or environmental stress.

A key feature of the system is the inclusion of an explanatory section that justifies the recommendation. It informs the user that the current growth stage and nutrient levels indicate a nitrogen deficiency, thereby improving transparency and trust in the system.

Overall, the Fertilizer Hub module provides a comprehensive, context-aware, and user-friendly approach to nutrient management, helping improve crop health, reduce input costs, and support sustainable farming practices.

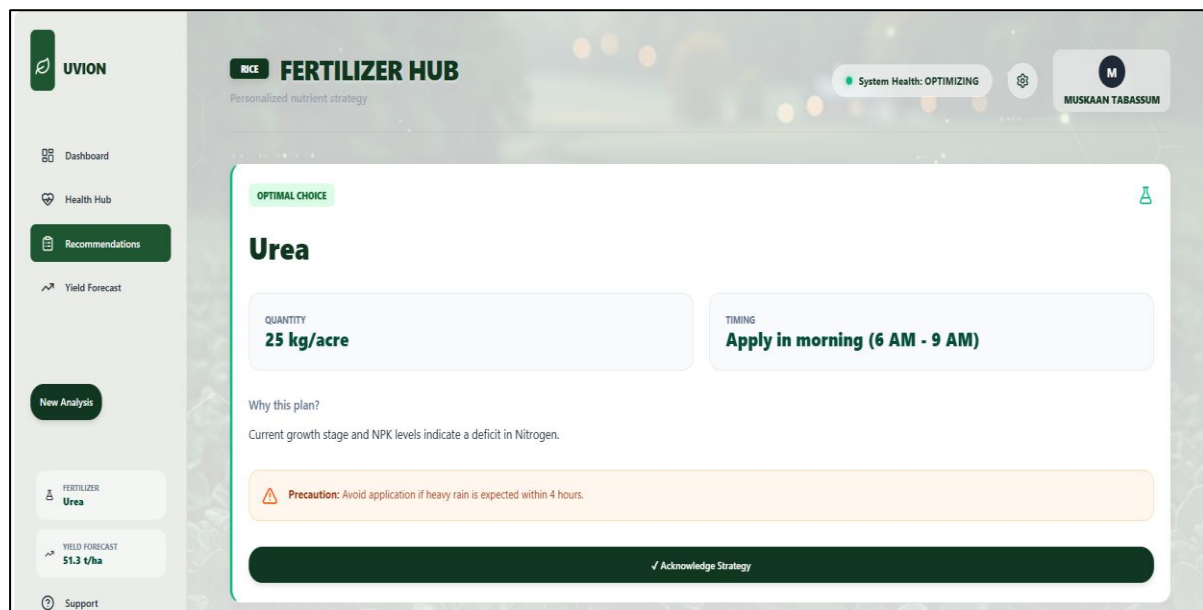


Fig 11.6 Fertilizer Recommendation

11.7 YEILD PREDICTION

The Yield Forecaster module is designed to estimate crop productivity and provide insights into potential yield outcomes under current and optimal conditions. It serves as a predictive analytics component that helps farmers understand both expected performance and improvement opportunities.

The interface displays the expected yield, which represents the predicted output based on current conditions such as soil quality, weather, disease status, and applied agricultural practices. In this case, the expected yield is shown as 51.3 tons per hectare, along with a comparative improvement of 4.2% from the previous cycle, indicating positive progress.

Alongside this, the system presents the potential yield, which represents the maximum achievable output under ideal conditions. This value, shown as 64.1 tons per hectare, serves as a benchmark for optimal productivity and helps in identifying the performance gap.

A critical feature of this module is the yield gap analysis section. It provides an evaluation of how close the current performance is to the maximum potential. The system indicates that the field is operating at approximately 80% of its capacity, highlighting room for improvement.

The module further identifies limiting factors that contribute to reduced yield. In this case, the system detects that suboptimal nitrogen levels in the soil are responsible for a 25% reduction in potential yield. This insight directly connects with the fertilizer recommendation module, enabling a cohesive decision-making process.

The user interface is designed to present this information clearly and concisely, allowing users to quickly understand their crop's performance status and take corrective actions. The integration with other modules ensures that yield predictions are continuously updated based on new data inputs and user actions.

Overall, the Yield Forecaster module transforms raw data into meaningful productivity insights, enabling farmers to make informed decisions, optimize resource usage, and maximize crop output.

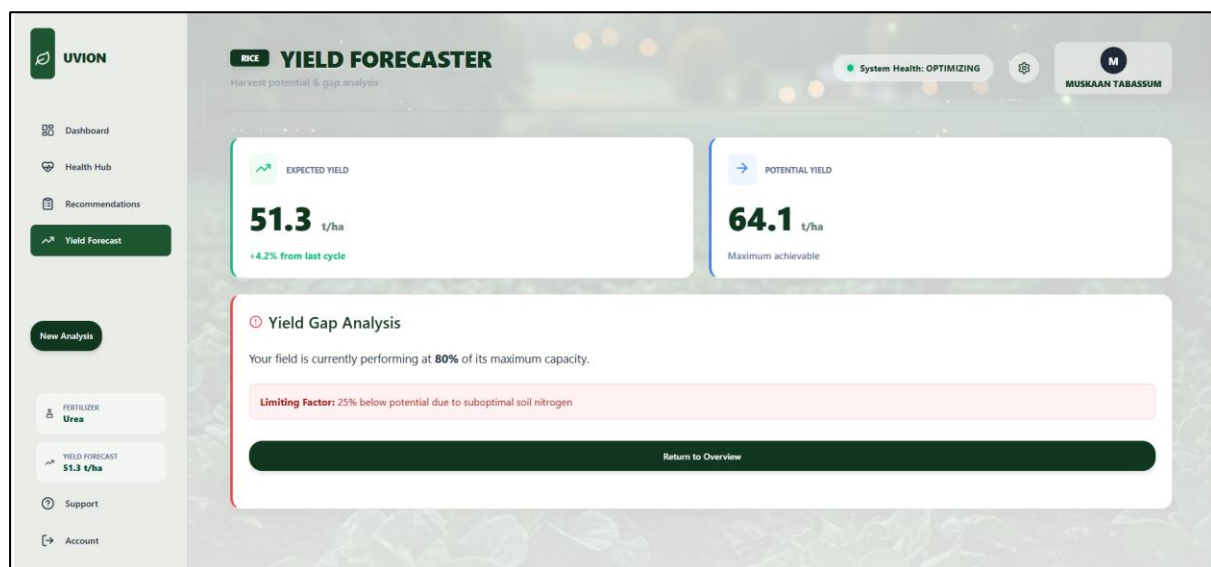


Fig 11.7 Yeild Prediction

CHAPTER 12

CONCLUSION

12.1 CONCLUSION

The UVION system is an intelligent smart farming solution that integrates multiple technologies such as machine learning, deep learning, and real-time data processing to assist farmers in making informed decisions. The project focuses on solving key agricultural challenges by combining modules like disease detection, yield prediction, fertilizer recommendation, crop recommendation, weather analysis, and growth monitoring into a single unified platform. Unlike traditional systems that operate independently, UVION provides an integrated approach where all modules collaborate to generate accurate and context-aware recommendations.

By leveraging machine learning techniques and structured datasets, UVION demonstrates the ability to provide accurate and timely recommendations that can enhance crop productivity and resource efficiency. The use of a centralized dashboard further simplifies access to critical agricultural insights, making the system practical and accessible for farmers.

Although the current implementation relies on simulated data and basic models, it establishes a strong foundation for scalable and real-world applications. With future enhancements such as real-time IoT integration, advanced deep learning models, and expanded crop support, the system has the potential to significantly contribute to sustainable and precision agriculture.

In conclusion, UVION serves as a promising smart farming solution that bridges the gap between technology and agriculture, aiming to improve productivity, reduce risks, and support informed decision-making for farmers.

12.2 FUTURE WORK

a. Real-Time IoT Sensor Integration

The system can be enhanced by integrating real-time IoT sensors to collect live field data such as soil moisture, NPK levels, temperature, and humidity. This will replace simulated data with actual environmental inputs, improving prediction accuracy and enabling real-time decision-making through edge computing.

b. Advanced Machine Learning and Deep Learning Models

Future improvements include upgrading existing models using deep learning architectures and ensemble techniques. Disease detection can be enhanced using real-world datasets,

while yield prediction and fertilizer recommendation modules can be optimized with adaptive and hybrid ML approaches for higher accuracy.

c. Expansion to Multi-Crop and Region-Specific Models

The system can be extended to support a wider variety of crops and regional variations. This includes incorporating crop-specific disease detection, fertilizer recommendations, and growth models to make the system more scalable and applicable across different agricultural environments.

d. Voice-Based and Multilingual Interaction

To improve accessibility, the system can include voice-based interaction and multilingual support. This will allow farmers to interact using regional languages and speech input, making the platform more user-friendly and inclusive for non-technical users.

REFERENCES

- [1] S. Sen and Mithlesh, "A Survey on Crop Disease Detection Using Image Processing Techniques," Semantic Scholar, 2020.
<https://www.semanticscholar.org/paper/A-SURVEY-ON-CROP-DISEASE-DETECTION-USING-IMAGE-FOR-Sen-Mithlesh/16a5051af73f9d5c5a7778e9b11b1b16f1cf19b4>
- [2] M. M. Islam, M. S. Rahman, and A. Ferdous, "Deep learning-based crop disease prediction with web applications," Array, Elsevier, 2023.
<https://www.sciencedirect.com/science/article/pii/S2666154323002715>
- [3] H. N. Ngugi, A. M. Mwangi, and M. W. Kamau, "Machine Learning and Deep Learning for Crop Disease Diagnosis in Sustainable Agriculture," Agronomy, MDPI, vol. 14, no. 12, pp. 1–22, 2024.
<https://www.mdpi.com/2073-4395/14/12/3001>
- [4] S. P. Mohanty, D. Hughes, and M. Salathé, "Plant Disease Detection Using Machine Learning," ResearchGate, 2018.
https://www.researchgate.net/publication/327065422_Plant_Disease_Detection_Using_Machine_Learning - Google Search
- [5] K. Ramu and K. Priyadarsini, "A Review on Crop Yield Prediction Using Machine Learning Methods," ICOSEC Conference, 2021.
https://www.researchgate.net/publication/356206189_A_Review_on_Crop_Yield_prediction_Using_Machine_Learning_Methods?utm_source=chatgpt.com
- [6] M. M. Islam, M. S. Rahman, and A. Ferdous, "Deep Learning-Based Crop Disease Prediction with Web Applications," Array, Elsevier, 2023.
<https://www.sciencedirect.com/science/article/pii/S2666154323002715>
- [7] S. Archana, "A Survey on Deep Learning Based Crop Yield Prediction," NEPT Journal, vol. 7, pp. 1–10, 2024.
<https://neptjournal.com/upload-images/%284%29B-3977.pdf>
- [8] A. Kumar, R. Singh, and P. Sharma, "Smart Farming with IoT and Machine Learning for Crop Recommendation and Disease Detection," International Journal for Research in Applied Science & Engineering Technology (IJRASET), 2023.
<https://www.ijraset.com/research-paper/smart-farming-with-iot-and-machine-learning-for-crop-recommendation-and-disease-detection>

- [9] M. Shafay, et al., “Recent advances in plant disease detection: challenges and prospects using deep learning and hyperspectral / RGB imaging,” *Plant Methods*, 2025.
<https://plantmethods.biomedcentral.com/articles/10.1186/s13007-025-01450-0>
- [10] S. M. Shawon, Falguny Barua Ema, Asura Khanom Mahi, et al., “Crop Yield Prediction Using Machine Learning: An Extensive and Systematic Literature Review (2017–2024),” 2024. <https://www.sciencedirect.com/science/article/pii/S2772375524003228>

APPENDIX 1

RELEVANCE OF PROJECT TO Cos/Pos/PSOs

Title of Project	UVION – AI-Powered Autonomous Farming System
Implementation Details	The UVION system is an AI-driven agricultural platform designed to assist farmers in making informed decisions by integrating three core functionalities: plant disease detection using deep learning, fertilizer recommendation using machine learning, and crop yield prediction based on environmental and soil data. The system processes image inputs and numerical data through trained models and provides actionable outputs through a user-friendly interface. The backend is developed using FastAPI, ensuring efficient API handling, while the frontend supports multilingual interaction for better accessibility.
Cost (hardware or software cost)	Rs. 16,200
Type (Application, Product, Research, Review, etc.)	The UVION system is categorized as an application-based project with research-oriented components.

Relevance		PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO 10	PO 11	PO 12	PSO1	PSO2
	CO1	3	3	2	2	3	1	1	1	2	2	1	2	3	3
	CO2	2	3	3	2	3	2	2	2	2	2	3	2	3	3
	CO3	2	2	3	2	2	1	2	1	3	3	3	2	3	2
	CO4	1	1	1	1	2	1	1	2	2	2	2	2	2	1

CO1–PO–PSO Mapping Table

S.No	Mapping	Relevance	Justification
1	CO1–PO1	3	Applies AI and engineering knowledge
2	CO1–PO2	3	Analyzes agricultural problems
3	CO1–PO3	2	Designs integrated smart system
4	CO1–PO4	2	Validates using experiments
5	CO1–PO5	3	Uses modern tools effectively
6	CO1–PO6	1	Addresses farmer needs
7	CO1–PO7	2	Promotes sustainable farming
8	CO1–PO8	1	Ensures ethical AI usage
9	CO1–PO9	2	Demonstrates teamwork
10	CO1–PO10	2	Communicates results clearly
11	CO1–PO11	1	Applies project planning
12	CO1–PO12	2	Adapts to new technologies
13	CO1–PSO1	3	Develops AI-based systems

S.No	Mapping	Relevance	Justification
14	CO1-PSO2	3	Performs data-driven decisions

CO2-PO-PSO Mapping Table

S.No	Mapping	Relevance	Justification
1	CO2-PO1	2	Applies analytical knowledge
2	CO2-PO2	3	Evaluates agricultural problems
3	CO2-PO3	3	Designs optimal solutions
4	CO2-PO4	2	Investigates model performance
5	CO2-PO5	3	Uses ML tools for evaluation
6	CO2-PO6	2	Considers farmer needs
7	CO2-PO7	2	Supports sustainable decisions
8	CO2-PO8	2	Ensures ethical outputs
9	CO2-PO9	2	Works collaboratively
10	CO2-PO10	2	Explains model results
11	CO2-PO11	3	Manages system workflow
12	CO2-PO12	2	Learns new approaches
13	CO2-PSO1	3	Builds AI solutions
14	CO2-PSO2	3	Performs data analysis

CO3-PO-PSO Mapping Table

S.No	Mapping	Relevance	Justification
1	CO3-PO1	2	Applies engineering basics
2	CO3-PO2	2	Identifies system issues
3	CO3-PO3	3	Develops system architecture
4	CO3-PO4	2	Tests system modules
5	CO3-PO5	2	Uses development tools
6	CO3-PO6	1	Addresses user requirements
7	CO3-PO7	2	Supports sustainability
8	CO3-PO8	1	Maintains ethical design
9	CO3-PO9	3	Demonstrates teamwork
10	CO3-PO10	2	Communicates system design
11	CO3-PO11	3	Applies project management
12	CO3-PO12	2	Adapts to technologies
13	CO3-PSO1	3	Implements AI systems
14	CO3-PSO2	2	Uses data for decisions

CO4-PO-PSO Mapping Table

S.No	Mapping	Relevance	Justification
1	CO4-PO1	1	Applies basic knowledge
2	CO4-PO2	1	Understands problems
3	CO4-PO3	1	Supports system design

S.No	Mapping	Relevance	Justification
4	CO4-PO4	1	Observes system results
5	CO4-PO5	2	Uses tools for presentation
6	CO4-PO6	1	Addresses user clarity
7	CO4-PO7	1	Promotes awareness
8	CO4-PO8	2	Ensures ethical communication
9	CO4-PO9	2	Works in team environment
10	CO4-PO10	3	Communicates effectively
11	CO4-PO11	2	Follows documentation process
12	CO4-PO12	2	Learns communication skills
13	CO4-PSO1	2	Presents AI solutions
14	CO4-PSO2	1	Explains data insights

Relevance scale:

1 → Low

2 → Medium

3 → High

Course outcomes	
CO1	Demonstrate the ability to synthesize and apply the knowledge and skills acquired in the academic program to real-world problems.
CO2	Evaluate different solutions based on economic and technical feasibility
CO3	Effectively plan a project and confidently perform all aspects of project management
CO4	Demonstrate effective written and oral communication skills

AI&DS-PSO1:

Design and deploy scalable, ethical AI and data-driven solutions using AI, ML, and Data Science across domains.

AI&DS-PSO2:

Analyse and transform complex data into actionable insights and reliable decision-making using modern tools and algorithms.

PROGRAM OUTCOMES (PO's)

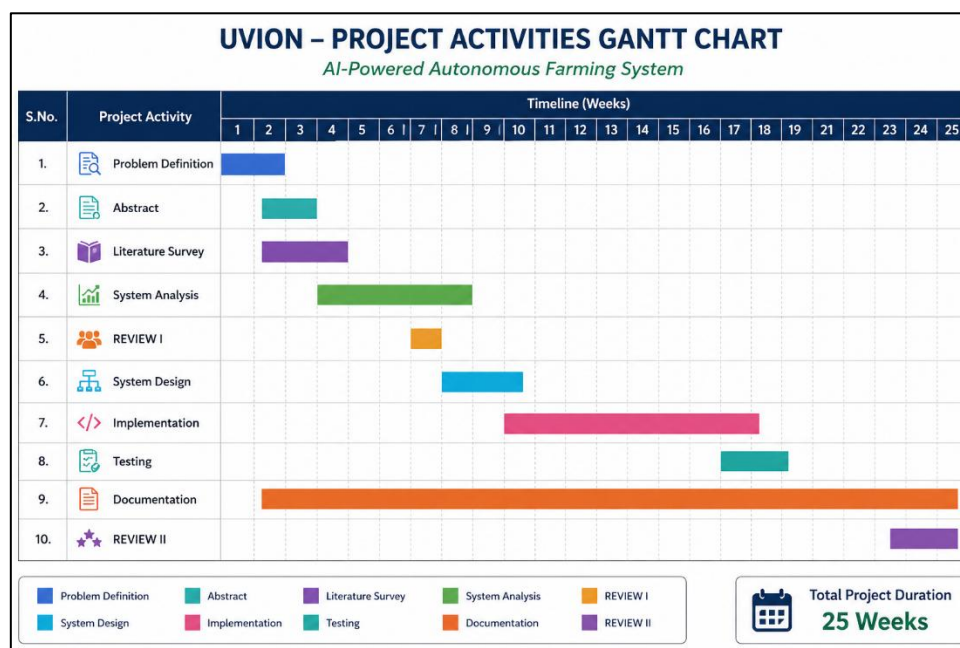
1. **Engineering knowledge:** Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.

2. **Problem analysis:** Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.

3. **Design/development of solutions:** Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.
4. **Conduct investigations of complex problems:** Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.
5. **Modern tool usage:** Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.
6. **The engineer and society:** Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.
7. **Environment and sustainability:** Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.
8. **Ethics:** Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.
9. **Individual and team work:** Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.
10. **Communication:** Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.
11. **Project management and finance:** Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.
12. **Life-long learning:** Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

APPENDIX 2

GANTT CHART



PROJECT ACTIVITIES DURATION CHART

PROJECT ACTIVITIES DURATION CHART – UVION						
UVION – AI-Powered Autonomous Farming System						
S.No.	Activity	Plan start (No. of weeks)	Plan duration (No. of weeks)	Actual start (No. of weeks)	Actual Duration (No. of weeks)	Percent complete
1.	Problem Definition	1	2	1	3	100%
2.	Abstract	3	1	3	1	100%
3.	Literature Survey	3	2	3	3	100%
4.	System Analysis	5	3	5	4	100%
5.	REVIEW I	8	1	8	1	100%
6.	System Design	9	2	9	3	100%
7.	Implementation	11	6	11	8	100%
8.	Testing	18	2	18	2	100%
9.	Documentation	3	24	6	23	100%
10.	REVIEW II	24	1	24	1	100%

Total Project Duration (Actual) : **25 Weeks**

Note: All activities are completed as per planned schedule.